# A PROJECT REPORT ON DEEP LEARNING BAESD INDIAN SIGN LANGUAGE GESTURE CLASSIFCATION USING CNN AND MEDIA PIPE

SUBMITTED TO
**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA**

In the partial fulfilment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY**
**IN**

**COMPUTER SCIENCE AND ENGINEERING**

Submitted by

**JAVVAJI VARALAKSHMI**

**22NG1A0599**

Under the Esteemed Guidance of

**S MOUNIKA**

Assistant Professor

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**USHARAMA**
**COLLEGE OF ENGINEERING AND TECHNOLOGY**
**AUTONOMOUS**

(Approved by AICTE and JNTUK, Kakinada)
(ON NH 16, TELAPROLU, NEAR GANNAVARAM - 521109)
**2025-2026**

USHARAMA
COLLEGE OF ENGINEERING AND TECHNOLOGY

**AUTONOMOUS**

## Vision of the Institute

To emerge as a Centre of excellence in technical education by imparting quality teaching learning practices and research for the transformation of society.

## Mission of the Institute

M1: Provide an ideal and the best class infrastructure to foster exploration in engineering and research

M2: Build dedicated faculty with student centric teaching, incorporating experiential, innovative skills.

M3: Encourage life-long learning, entrepreneurial thinking, and ethical responsibility in students to address societal challenges.

USHARAMA
COLLEGE OF ENGINEERING AND TECHNOLOGY

AUTONOMOUS

## DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

### Vision of the Department

To emerge as a skilled Technocrats on global scale in Computer Science and Engineering through quality education, innovation, collaborative researchers and entrepreneurs with moral values.

### Mission of the Department

**M1:** To impart quality education to the students.
**M2:** To pursue creative research and new technologies in Computer Science and Engineering.
**M3:** To encourage entrepreneurship skills among students and inculcate moral and ethical values to serve society.

### Department Program Educational Objectives (PEOs)

**PEO1:** Our graduates will establish themselves as effective professionals in industry, academia and entrepreneurship.
**PEO2:** Our graduates will become profound researchers in multiple domains.
**PEO3:** Our graduates will act as leaders in society.

### Department Program Specific Outcomes (PSOs)

**PSO1:** Illustrating a comprehensive understanding of fundamental computer system principles encompassing both hardware and software components to cultivate strong conceptual skills in processing and assigning computation solutions.

**PSO2:** Demonstrate and design proficient and technical abilities in algorithms, networking, web design, cloud computing and data analytics enabling the development of innovative solutions to complex real- world problems while identifying and addressing emerging research gaps

## Program Outcomes (POs)

**PO1**: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization.

**PO2**: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

**PO3**: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

**PO4**: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

**PO5**: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

**PO6**: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

**PO7**: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

**PO8**: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

**PO9**: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

**PO10**: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

**PO11**: Life-Long Learning: Recognize the need for and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

# USHARAMA
## COLLEGE OF ENGINEERING AND TECHNOLOGY

**AUTONOMOUS**

## DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

## CERTIFICATE

This is to certify that this Project Work entitled **"DEEP LEARNING BASED INDIAN SIGN LANGUAGE GESTURE CLASSIFICATION USING CNN AND MEDIA PIPE"** is the Bonafide work of **J. VARALAKSHMI (22NG1A0599)** who carried out the work under my supervision and submitted it in partial fulfilment of the requirements for the award of the degree in Bachelor of Technology in Computer Science and Engineering during the academic year **2025-2026.**

**Project Guide**
**S MOUNIKA**
Assistant Professor

**Head of the Department**
**Dr S. M. ROY CHOUDRI**

**Signature of the External Examiner**

# DECLARATION

I hereby declare that the Project Work entitled **"DEEP LEARNING BASED INDIAN SIGN LANGUAGE GESTURE CLASSIFICATION USING CNN AND MEDIA PIPE"** is the work done by me during the academic year **2025-2026** and is submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **COMPUTER SCIENCE AND ENGINEERING** from **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA**.

**BY**

**J. VARALAKSHMI**

**22NG1A0599**

# ACKNOWLEDGEMENT

I am pleased to acknowledge my sincere thanks to our Honorable Chairman **SRI. S. RAMABRAHMAM** for the support and encouragement that is given, and for providing sufficient resources.

I wish to avail ourselves of this opportunity to express our thanks to **Dr G. V. K. S. V PRASAD**, Principal, URCE, for his continuous support and giving valuable suggestions during the entire period of the Project Work.

I am taking this opportunity to express my gratitude to **Dr S. M. ROY CHOUDRI**, Head of the Department of Computer Science and Engineering, and my guide **S MOUNIKA**, Assistant Professor in Computer Science and Engineering, for their valuable support and motivation at every point in the successful completion of the Project Work.

I also place my floral gratitude to all other teaching staff and lab technicians for their constant support and advice throughout the Project Work.

**BY**

**J. VARALAKSHMI**

**22NG1A0599**

# DEEP LEARNING BASED INDIAN SIGN LANGUAGE GESTURE CLASSIFICATION USING CNN AND MEDIA PIPE

# ABSTRACT

This project presents a **real-time Indian Sign Language (ISL) Recognition System** designed to identify hand gestures representing digits and letters using deep learning. The system integrates a **Convolutional Neural Network (CNN)** trained on ISL gesture images with **MediaPipe Hands**, which performs accurate hand detection and cropping before classification. By isolating the hand region through MediaPipe, the system significantly improves prediction quality, reduces background noise, and enhances robustness for real-time webcam input. The preprocessing pipeline includes gesture detection, image resizing, normalization, and converting the input into a suitable format for the CNN model.

The application is deployed using **HTML,CSS,JS** offering a simple and interactive interface with two modes: **image upload**. Once the user provides an image, the system processes the hand region and predicts the corresponding ISL gesture along with confidence scores and class-wise probability distribution. The modular structure of the code ensures efficient model loading, fast inference, and real-time performance.

This ISL recognition system supports inclusivity by providing an automated tool to bridge communication gaps between the deaf community and the hearing population. Its lightweight architecture, real-time detection capability, and ease of deployment make it suitable for assistive communication tools, educational environments, and gesture-based human-computer interaction applications.

**Keywords:** Indian Sign Language (ISL), Sign Gesture Recognition, Convolutional Neural Network (CNN), MediaPipe Hands Detection, Deep Learning, Real-Time Image Classification, Hand Gesture Processing, HTML Application, Assistive Communication Technology.

# TABLE OF CONTENTS

## LIST OF FIGURES

## LIST OF ABBRIVATIONS

| | |
|---|---|
| **CNN** | Convolutional Neural Network |
| **ISL** | Indian Sign Language |
| **LSMT** | Long Short-Term Memory |
| **SVM** | Support Vector Machine |
| **PCA** | Principal Component Analysis |
| **EEG** | Electroencephalography |
| **FFT** | Fast Fourier Transform |
| **WGAN** | Wasserstein Generative Adversarial Network |
| **RNN** | Recurrent Neural Network |
| **GRU** | Gated Recurrent Unit |
| **TTS** | Text-to-Speech |

# CHAPTER 1
# INTRODUCTION

Gesture recognition has become one of the most promising fields in human–computer interaction, especially for communication systems designed for individuals with hearing and speech impairments. Sign languages, such as American Sign Language (ASL) or Indian Sign Language (ISL), rely heavily on hand shapes, finger positions, and movements to convey meaning. Recognizing these gestures automatically through computer vision provides a foundation for building intelligent assistive technologies. The project illustrated above presents a complete workflow for gesture recognition, beginning from raw gesture input and progressing through advanced image processing, feature extraction, learning, and final classification.

The system starts by capturing **hand gesture images** representing different signs. These images act as the primary input and must clearly show the hand configuration to ensure accurate recognition. Once collected, the images undergo a **pre-processing phase** where noise is removed, the background is suppressed, and the hand region is segmented. Techniques such as grayscale conversion, thresholding, edge detection, and morphological operations help transform the input into a cleaner and more meaningful representation. Pre-processing enhances the important visual features of the gesture, reduces computational complexity, and prepares the images for the next stage.

Following pre-processing, the system moves into **feature extraction and selection**, which is one of the most crucial phases in the recognition pipeline. The purpose of feature extraction is to capture the distinctive characteristics of each hand gesture. These may include edge patterns, contour shapes, finger outlines, distances between landmarks, or geometric shapes of the hand pose. Effective feature extraction ensures that the classifier receives structured numerical information rather than raw pixel data. Feature selection then reduces unnecessary or redundant information, retaining only the most relevant features that contribute to accurate gesture classification. This improves both the speed and performance of the model.

Once the features are prepared, the data is fed into a **classification framework**, where machine learning or deep learning algorithms learn to differentiate between various gesture categories.

During the **training phase**, the classifier learns patterns from labeled gesture samples. Algorithms such as Convolutional Neural Networks (CNNs), Support Vector Machines (SVM), Artificial Neural Networks (ANN), or hybrid models (e.g., CNN–LSTM) are commonly used for this task. The classifier is then evaluated using unseen gesture images in the **testing phase**, ensuring that it can generalize to new inputs. A robust classifier should achieve high accuracy, low error rates, and consistent performance across different gesture types.

Finally, the system generates an **output**, which corresponds to the recognized gesture. This output may represent an alphabet letter, a word, or a command, depending on the application. The recognized gesture can be displayed as text, converted to audio, or used to control other systems. Such recognition platforms play a vital role in assistive technology by enabling communication between hearing-impaired individuals and the general public. They also support education, virtual.

Overall, this project demonstrates a complete and systematic approach to gesture recognition from image acquisition to pre-processing, feature extraction, classification, and output generation. By combining computer vision techniques with powerful learning algorithms, the system offers an intelligent and scalable solution for automatic sign language recognition. With further advancements in deep learning and real-time processing, such systems can be expanded to handle dynamic gestures, continuous signing, and multilingual sign languages, making them even more accessible and effective for real-world use.

Gesture recognition has emerged as a transformative technology in the field of human–computer interaction, especially for applications involving communication with individuals who rely on sign language. Traditional methods of communication often create barriers for people with hearing or speech impairments, as most hearing individuals are not trained in sign language. Thus, the development of automated sign recognition systems can significantly enhance accessibility and foster inclusivity. In this context, computer vision and artificial intelligence provide efficient solutions for interpreting hand gestures and translating them into meaningful information. The gesture recognition system illustrated in the figure represents a complete computational workflow—from the acquisition of hand gesture images to the prediction of gesture classes using advanced machine learning techniques. This detailed explanation outlines every stage of the system, describing how raw gesture images are transformed into accurate classification results.

The system begins with the **input phase**, where gesture images representing American Sign Language (ASL) or Indian Sign Language (ISL) hand shapes are captured. These images can be acquired using a standard camera, webcam, or through pre-existing gesture datasets. The quality, clarity, and visibility of the hand shape in the captured images are essential for efficient recognition. In many real-world scenarios, input gestures may vary due to lighting conditions, background variations, camera angles, and occlusions. Because of this natural variability, the system must be robust enough to handle inconsistent environments. The input gesture image forms the foundation for the subsequent stages, serving as the raw material from which the system extracts meaningful information.

After obtaining the gesture image, the next step is **gesture image acquisition**, which focuses on isolating the relevant region of interest—the hand gesture—from the rest of the scene. This stage may include cropping or centering the hand region to improve visibility and reduce unnecessary background information. Gesture acquisition ensures that the recognition process begins with a well-focused representation of the hand. In some implementations, skin detection algorithms or bounding-box detectors are used to identify the hand region automatically. This step reduces noise and prepares the image for deeper analytical processing.

The **pre-processing** stage plays a crucial role in enhancing the gesture image and preparing it for feature extraction. Raw gesture images often contain noise due to environmental interference, inconsistent lighting, or camera limitations. Pre-processing aims to transform the image into a format that highlights essential features and suppresses irrelevant details. Common pre-processing techniques include converting the image to grayscale, removing background clutter, applying edge detection algorithms (such as Canny or Sobel filters), and performing thresholding operations to convert the image into a high-contrast binary form. These operations help emphasize boundaries and contours of the hand gesture, which are essential for recognition. In addition, morphological operations like dilation and erosion can be applied to reduce distortions and fill small gaps detected Once the image is pre-processed, the system proceeds to **feature extraction and selection**. Feature extraction is arguably the most significant phase in gesture recognition because it transforms visual information into numerical representations that machine learning models can interpret. In gesture recognition, features may include geometric shapes, contour outlines, finger positions, curvature points, angles between hand joints, or pixel intensity patterns. For example, edge detection results

highlight the outline of the gesture, enabling the extraction of shape-based features. Some methods extract skeleton representations of the hand, identifying key nodes corresponding to fingertips and joints. Others rely on keypoint detection algorithms, such as those found in MediaPipe or OpenPose, to capture the spatial structure of the hand. The extracted features serve as a compact description of the gesture, representing its unique characteristics in a mathematical form.

Feature selection is equally important because not all features contribute to accurate gesture classification. Redundant or noisy features may decrease performance by confusing the classifier or increasing the computational complexity. Feature selection algorithms—such as Principal Component Analysis (PCA), correlation filtering, or mutual-information-based selection—help identify the most relevant features. By keeping only the essential information, the system becomes faster, more accurate, and less prone to overfitting. Feature extraction and selection collectively form the backbone of the system's recognition capability, converting raw gesture images into structured data suitable for machine learning.

The next phase of the workflow is **classification**, where the extracted features are used to train a machine learning or deep learning model. Classification in gesture recognition typically involves two major stages: training and testing. During the **training phase**, the classifier learns patterns from a set of labeled gesture images. Each image corresponds to a specific gesture class, such as "A," "B," "C," or other sign language symbols. Using these labeled examples, the classifier develops an understanding of the unique characteristics associated with each class. Deep learning models, such as Convolutional Neural Networks (CNNs), are particularly effective due to their ability to learn complex, hierarchical representations directly from data. They can automatically identify discriminative features such as finger shapes, spatial patterns, and edge textures. In some advanced implementations, hybrid models like CNN–LSTM are used, where the CNN extracts spatial features and the LSTM analyzes sequential patterns found in dynamic gestures or videos.

In the **testing phase**, unseen gesture images are fed into the classifier to evaluate its generalization capability. High testing accuracy indicates that the model is not overfitted and can correctly identify new gesture samples. Classification performance may be measured using metrics like accuracy, precision, recall, F1-score, and confusion matrices. These metrics provide insight into how well the model differentiates between similar gestures and handles variations in input images.

The final stage of the system is the **output generation**, where the recognized gesture is displayed or used for further processing. The output typically takes the form of a predicted gesture label, such as an alphabet letter or word. In real-world applications, the recognized gesture may be converted to speech, enabling communication between hearing-impaired individuals and people who cannot read sign language. The output can also be integrated into gesture-controlled interfaces, virtual reality environments, or robotic systems. The simplicity and versatility of this output stage make gesture recognition systems highly adaptable to different use cases.

The gesture recognition system illustrated in the diagram demonstrates a systematic and well-organized approach that combines image processing, feature engineering, machine learning, and classification strategies. By breaking down the problem into smaller stages—acquisition, pre-processing, feature extraction, classification, and output—the system ensures reliability, consistency, and accuracy. Moreover, this modular design allows for flexibility and scalability. For instance, more advanced feature extraction methods, such as deep convolutional feature maps, can replace traditional edge-based techniques. Similarly, more powerful classifiers, such as transformers or attention-based networks, can be integrated to further enhance performance.

Overall, the described gesture recognition framework provides a strong foundation for building practical applications that interpret sign language gestures and bridge communication gaps. With continued advancements in artificial intelligence, real-time detection, and low-cost hardware, such systems can evolve into fully interactive platforms capable of understanding continuous sign language, multi-hand gestures, and even facial expressions that accompany manual signs. As a result, the project contributes significantly to creating inclusive technologies that empower individuals with hearing and speech impairments, promote accessibility, and enable seamless human–machine interaction.

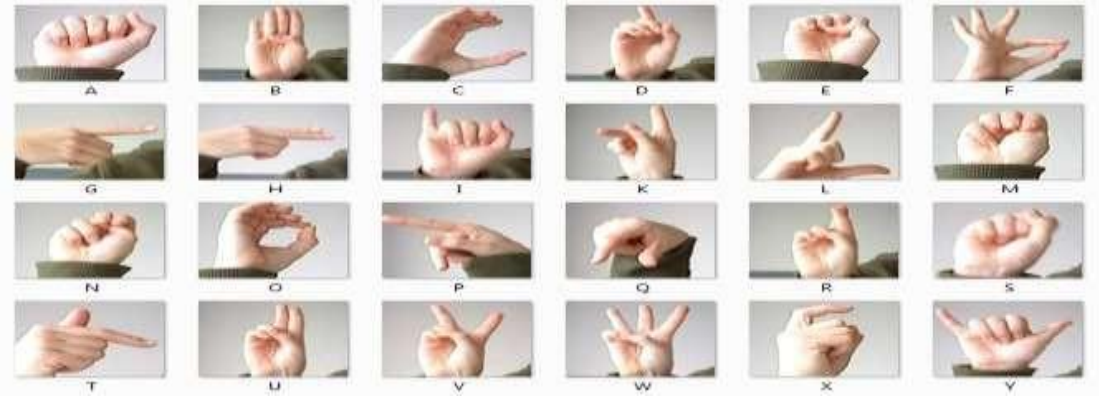


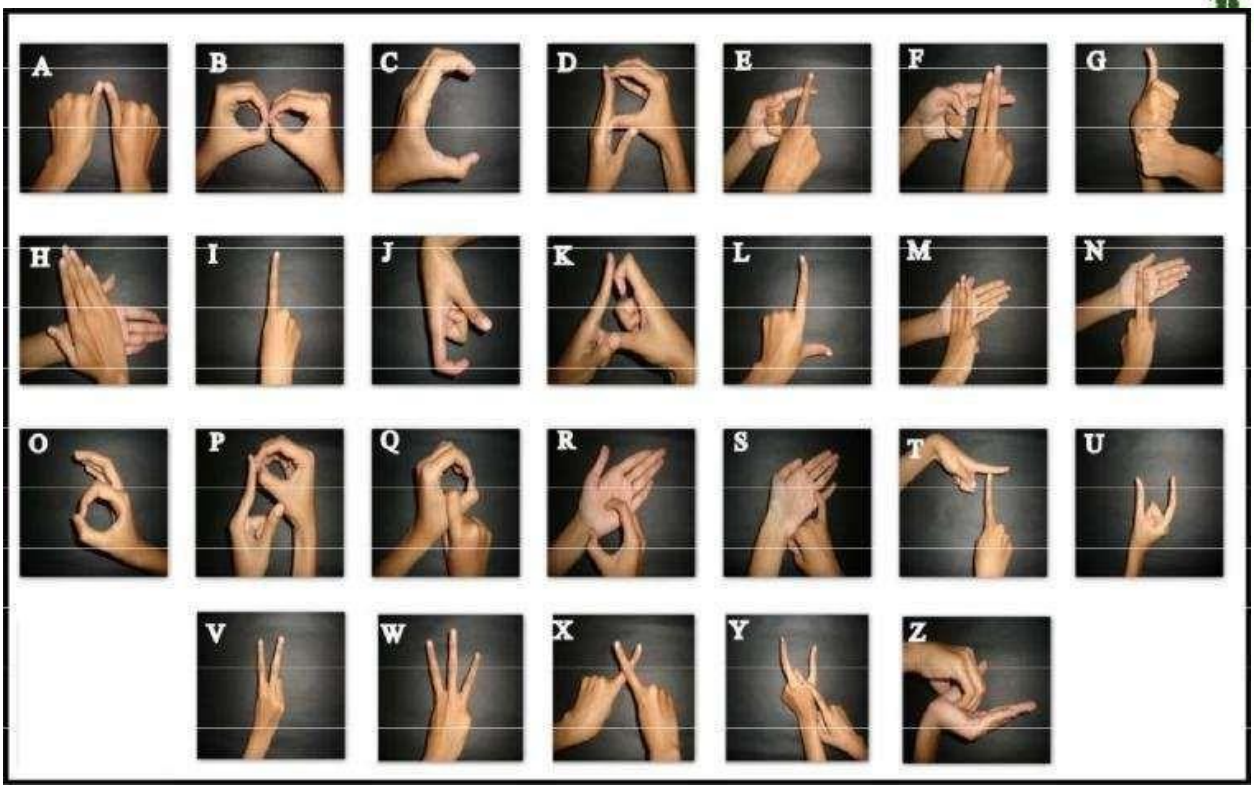

**Figure 1: ISL Hand Gesture Alphabet**

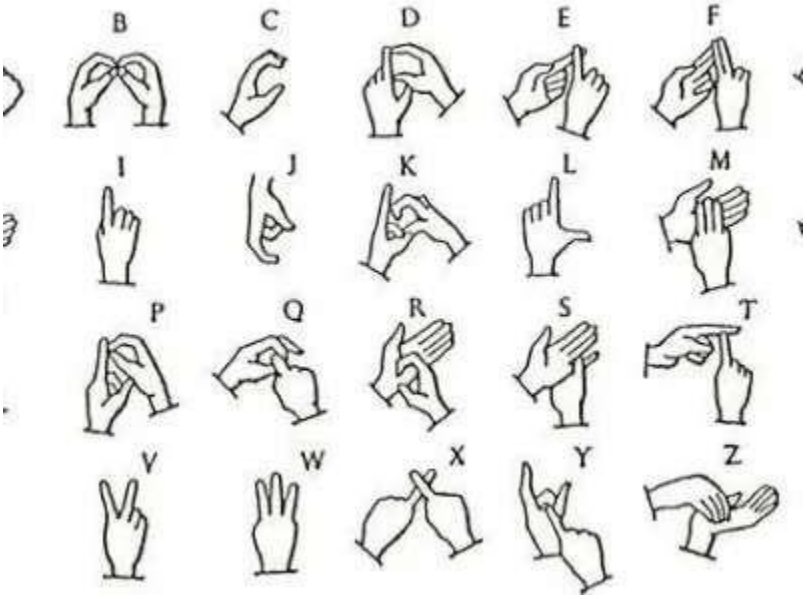
The following image represents a set of **static hand gestures** used in **Indian Sign Language (ISL)** to denote the English alphabet (A–Y). Each gesture corresponds to a unique hand shape, finger position, and orientation, allowing users to visually communicate letters without speech. Static gestures are widely used in finger-spelling, where words are spelled out letter by letter using distinct hand signs.

These alphabet gestures form the **foundation of ISL communication**, especially when expressing names, proper nouns, abbreviations, or vocabulary that does not have a dedicated gesture. In gesture recognition research, such static signs serve as the primary dataset for training deep learning models because:

- They have **clear and consistent shapes**,
- They can be captured using simple camera input,
- They allow easy classification with CNN models, and
- They help the system learn precise finger positions and hand contours.

In the image below, each box shows a cropped hand region displaying the ISL pose for a corresponding alphabet letter:


A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Y
Z


B
C
D
E
F
I
J
K
L
M
P
Q
R
S
T
V
W
X
Y
Z


These alphabet gestures (A–Y) provide the visual patterns used to train classification models for **Sign Language Recognition**. Deep learning models—particularly Convolutional Neural Networks (CNNs)—analyze these gestures by learning the shape, orientation, and structure of the fingers.

In this project, similar gesture images are preprocessed using **MediaPipe Hands** to automatically detect, crop, and normalize the hand region before classification. This enhances accuracy by removing background noise and focusing only on the hand shape.

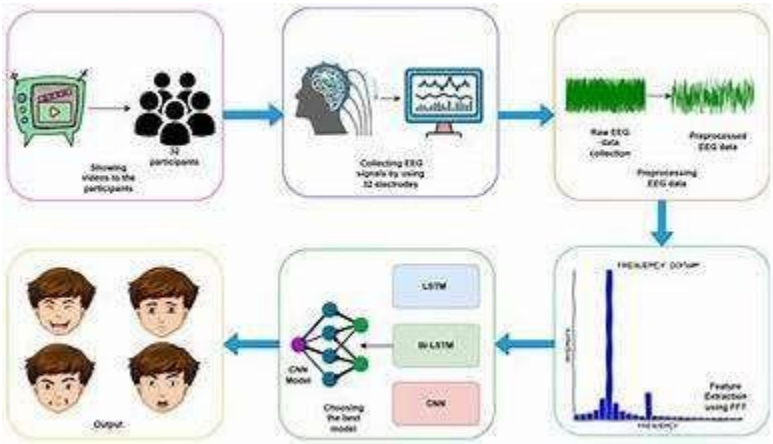



**Figure 2: Overview of the EEG-Based Emotion Recognition Workflow**

The above figure illustrates the complete workflow of an **EEG-based emotion recognition system**, where raw brain signals are collected, preprocessed, analyzed, and classified using deep learning models to determine the emotional state of participants.

The process consists of the following major stages:

**1. Stimulus Presentation**

Participants are shown visual stimuli such as videos, images, or emotional content designed to trigger various emotional states. This step ensures natural emotional responses, making the EEG recordings rich and meaningful.

**2. EEG Signal Acquisition**

Brain activity is recorded from participants using an EEG headset equipped with multiple channels (e.g., 32 electrodes). These electrodes capture electrical impulses generated by neuronal activity, providing real-time data reflecting the participant's emotional state.

**3. Preprocessing of Raw EEG Data**

The raw EEG signal often contains noise from eye movement, muscle activity, and environmental interference. Therefore, preprocessing is performed, which includes:

- Noise removal and filtering
- Signal normalization
- Segmentation into usable frames

This step ensures that the data fed into the deep learning models is clean and reliable.

**4. Feature Extraction (Frequency Domain)**

The cleaned EEG signal undergoes feature extraction, typically using:

- Fast Fourier Transform (FFT)

- Wavelet Transform
- Power Spectral Density

  These methods convert the time-based signals into the frequency domain, where emotional patterns become more distinguishable. Extracted features represent critical frequency bands such as alpha, beta, gamma, delta, and theta waves.

**5. Deep Learning-Based Emotion Classification**

The extracted features are fed into different deep learning models, such as:

- **LSTM (Long Short-Term Memory)** – captures temporal dependencies
- **Bi-LSTM (Bidirectional LSTM)** – processes the signal in forward and backward directions
- **CNN (Convolutional Neural Network)** – learns spatial and spectral features

These models analyze patterns within the EEG signals and classify them into predefined emotional categories.

**6. Output: Emotion Prediction**

Finally, the system outputs the detected emotional state, which may include:

- Happy
- Sad
- Neutral
- Angry
- Surprised

These predictions reflect how the participant emotionally responded to the stimuli shown in the first stage.

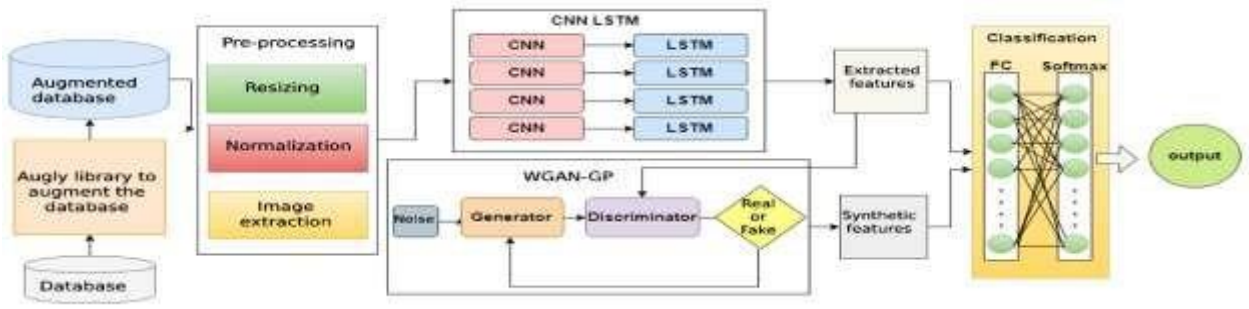



**Figure 3: Overall Architecture of the CNN–LSTM Model with WGAN-GP and Data Augmentation**

The above diagram presents a complete pipeline for building a robust deep-learning model that integrates **data augmentation**, **CNN–LSTM feature extraction**, **WGAN-GP synthetic data generation**, and **final classification**. This hybrid architecture enhances both the size and the quality of the training dataset, leading to improved model accuracy and generalization.

**1. Database Creation and Augmentation**

**Original Database**

The process begins with a base dataset containing raw images or signals. These samples may be insufficient for training deep neural networks because:

- The dataset may be small
- It may not include enough variation
- It may suffer from class imbalance

**Augly Library for Data Augmentation**

The **Augly** library is used to artificially increase the dataset size by applying:

- Rotation
- Cropping
- Scaling
- Color changes
- Noise injection

- Blurring

This step produces an **augmented database** with greater diversity, which helps prevent overfitting and improves the robustness of the model.

**2. Pre-processing Stage**

The augmented dataset is passed through a pre-processing pipeline consisting of:

**a. Resizing**

All images are resized to a fixed resolution so that the CNN receives uniform input dimensions.

**b. Normalization**

Pixel values are normalized (e.g., scaled between 0 and 1) to:

- Improve training speed
- Ensure numerical stability
- Enhance feature extraction quality

**c. Image Extraction**

Any additional processing such as segmentation, region-of-interest extraction, or color channel selection is applied here.

This pre-processing ensures that both real and augmented samples are clean, standardized, and ready for deep learning.

**3. CNN–LSTM Feature Extraction Block**

This hybrid block combines the strengths of CNNs and LSTMs:

**a. Convolutional Neural Networks (CNN)**

Multiple CNN layers extract spatial features such as:

- edges

- shapes
- textures
- fine-grained patterns

Each CNN block outputs high-level spatial features.

**b. Long Short-Term Memory (LSTM)**

The CNN features are fed into LSTM units to learn:

- sequential dependencies
- temporal relationships
- multi-frame continuity (if video/time-series data)

Both spatial and temporal features are jointly captured, resulting in a powerful representation of the input data.

**Extracted Features**

The output of the CNN–LSTM block forms the **feature vector** used for classification.

**4. WGAN-GP (Wasserstein GAN with Gradient Penalty) for Synthetic Feature Generation**

To further improve the dataset, a **WGAN-GP** model is included.

**a. Generator**

Takes random **noise** as input and produces synthetic feature samples that mimic real data.

**b. Discriminator**

Attempts to distinguish whether a sample is:

- Real (from CNN–LSTM) or
- Fake (from Generator)

**c. Gradient Penalty**

Ensures stable GAN training and avoids mode collapse.

The WGAN-GP block outputs **synthetic features**, which are combined with real features to create a richer training set.

**5. Classification Layer (FC + Softmax)**

The final classification module contains:

**a. Fully Connected (FC) Layers**

These dense layers perform nonlinear transformation of extracted or synthetic features.

**b. Softmax Layer**

Generates a probability distribution over possible classes.

**Final Output**

The system outputs the predicted category (e.g., class label, emotion, gesture, object type), completing the recognition pipeline.

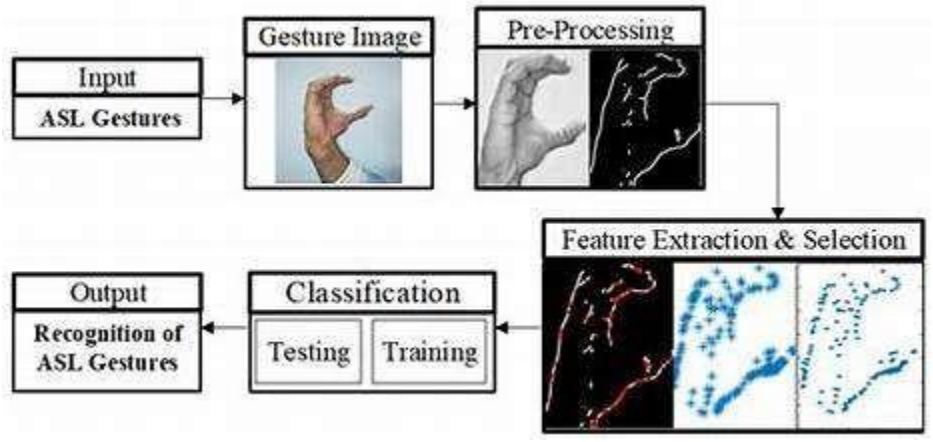



**Figure 4: Workflow of an ASL Gesture Recognition System**

The above figure illustrates the complete workflow for recognizing **American Sign Language (ASL) gestures** using image processing and machine learning techniques. The system processes gesture images step-by-step—from input to classification—ensuring accurate mapping of hand shapes to their corresponding sign meanings.

**1. Input: ASL Gesture Image**

The process begins with capturing an image of a hand gesture representing an ASL symbol. These images are typically acquired using:

- a webcam,
- a digital camera, or
- a dataset of pre-collected ASL gesture images.

The quality of this input image significantly influences the accuracy of recognition.

**2. Gesture Image Acquisition**

In this stage, the hand gesture is isolated as the primary region of interest. The goal is to ensure that the hand shape is clearly visible and well-centered for further processing. The acquired gesture image represents the raw data on which the system performs segmentation and feature learning.

**3. Pre-processing**

Pre-processing plays a crucial role in enhancing the gesture image and preparing it for feature extraction. Typical pre-processing operations include:

**a. Noise Removal**

Filters remove unwanted artifacts or background noise.

**b. Image Segmentation**

Separates the hand region from the background, often using:

- thresholding,

- edge detection,
- contour detection, or
- skin-color segmentation.

**c. Binarization**

Converts the gesture image into a binary or edge-detected format to highlight the hand shape.

These steps ensure that only relevant parts of the image are fed into subsequent stages, improving recognition accuracy.

**4. Feature Extraction & Selection**

This step extracts important visual features from the pre-processed image. Features may include:

- contour details
- edges of fingers
- geometric shape information
- keypoints
- skeleton representations

The purpose is to convert raw image data into a structured, numerical format that the classification model can understand.

**Feature Selection**

Not all extracted features contribute equally. This step chooses the most discriminative features to:

- reduce dimensionality,
- speed up computation, and
- improve model accuracy.

**5. Classification (Training & Testing)**

Once features are extracted, they are used to train machine learning or deep learning models. Common classifiers include:

- CNN-based models,
- SVM,
- k-NN,
- ANN, etc.

**Training Phase**

The model learns patterns from labeled ASL gesture samples.

**Testing Phase**

New, unseen images are used to evaluate how well the model generalizes. High accuracy in this stage indicates strong recognition capability.

**6. Output: Recognition of ASL Gestures**

The final output is the **predicted ASL gesture**, mapped to its corresponding alphabet or sign. This allows the system to interpret hand gestures into readable text or signals.

Examples of outputs include:

- "A", "B", "C", …
- words or commands depending on the application.

**1.1 Objective of the Project**

The primary objective of this project is to design and develop an efficient **Sign Language Gesture Recognition System** using deep learning and computer vision techniques. The system aims to interpret static hand gestures from sign languages (ASL/ISL) and classify them accurately into predefined categories. Through the use of preprocessing, feature extraction, and classification models, the project seeks to transform gesture images into meaningful outputs that can support communication for individuals with hearing or speech impairments.

**Specific Objectives:**

1. To capture and process gesture images representing different sign language symbols.
2. To perform image preprocessing such as resizing, normalization, and noise removal for improved recognition accuracy.
3. To extract meaningful features from gesture images using edge detection and shape-based or deep-learning-based approaches.
4. To classify gestures using machine learning or deep learning models such as CNN, SVM, or hybrid architectures.
5. To develop a system capable of recognizing gestures with high accuracy and reliability.
6. To contribute to assistive technology that bridges communication gaps between deaf individuals and the general population.

### 1.2 Software and Hardware Requirements

The following are the essential hardware and software components required for the development and execution of the Sign Language Gesture Recognition System.

**Hardware Requirements**

1. **Processor:** Intel i5/i7 or equivalent (minimum 2.0 GHz)
2. **RAM:** 8 GB or higher recommended
3. **Storage:** Minimum 20 GB available space
4. **Camera/Webcam:** HD webcam for real-time gesture capture
5. **GPU (Optional but recommended):** NVIDIA GPU for faster training of deep learning models
6. **Input Devices:** Keyboard, mouse
7. **Display:** Monitor with 1366×768 or above resolution

**Software Requirements**

1. **Operating System:** Windows / Linux / macOS
2. **Programming Language:** Python 3.8+
3. **Deep Learning Libraries:**

a. TensorFlow / Keras
b. PyTorch (optional)

4. **Computer Vision Libraries:**
    a. OpenCV
    b. MediaPipe (if hand landmark detection is used)
5. **Machine Learning Libraries:**
    a. Scikit-learn
    b. NumPy
    c. Pandas
6. **Development Tools:**
    a. Jupyter Notebook / PyCharm / VS Code
7. **Additional Libraries:**
    a. Matplotlib, Seaborn (for visualization)
    b. Augmentation libraries (e.g., AugLy, imgaug)
8. **Dataset:**
    a. ASL/ISL gesture image dataset

### 1.3 Modules Description

The proposed system for Indian Sign Language (ISL) recognition is divided into several logical modules. Each module performs a specific task and passes its output to the next stage, forming a complete pipeline from input image to final prediction. This modular design makes the system easier to understand, implement and extend. The main modules are: (i) Dataset and Input Acquisition Module, (ii) Hand Detection and Pre-processing Module, (iii) CNN-Based Classification Module, (iv) Application Interface Module, and (v) Prediction and Result Visualization Module. Together, these modules implement an end-to-end solution that can recognize ISL hand signs from both uploaded images and webcam input using a deep learning

model deployed through a Streamlit application.

### 1.3.1 Dataset and Input Acquisition Module

The first module is responsible for providing the input data to the system. During the model development phase, this module deals with the **dataset of ISL gesture images** that is used for training and testing the CNN model. These images represent different ISL digits and letters, captured under various conditions and stored in class-wise folders. Each image is associated with a label indicating which sign it corresponds to, such as "0–9" or "A–Z". This labeling makes supervised learning possible and allows the CNN to learn the mapping between image patterns and sign classes.

In the deployment phase, this same module also handles **live user input**. In your project, the Streamlit application provides two modes: *Upload Image* and *Use Webcam*. In the Upload mode, the user selects a hand sign image (in JPG or PNG format) from their local system, and the application reads it using the PIL library. In the Webcam mode, the user captures an image directly from the camera using the st.camera_input() component. In both cases, the output of this module is a raw RGB image that contains the hand performing an ISL gesture and is ready to be passed to the pre-processing module.

### 1.3.2 Hand Detection and Pre-processing Module

The second module is the **Hand Detection and Pre-processing Module**, which plays a crucial role in improving the quality of input before it reaches the classifier. In many real-world images, the background may contain objects, noise or other irrelevant details that can confuse the model. To solve this, your project uses **MediaPipe Hands** to automatically detect the hand region in the image. MediaPipe analyzes the input frame, identifies hand landmarks and computes their coordinates. Using these coordinates, a tight bounding box is drawn around the hand area, and this region is cropped from the original image. If MediaPipe fails to detect a hand, a fallback center crop is used so that the system remains robust even in difficult cases.

**Main**

Once the hand is cropped, the pre-processing pipeline standardizes the image for the CNN. First,

the cropped hand image is resized to a fixed resolution, such as 64×64 pixels, which matches the input size used during model training. This ensures that all images, regardless of their original dimensions, become compatible with the model. Then, the pixel values are converted to floating-point numbers and normalized to a range between 0 and 1 by dividing by 255.0. Normalization helps the network train and predict more efficiently and avoids numerical instability. If the input image is grayscale or has a different channel arrangement, it is converted to a 3-channel RGB format to maintain consistency with the model's expectations. Finally, a batch dimension is added to the image array so that it has the shape required by Keras/TensorFlow models. The output of this module is a clean, normalized, cropped hand image ready for classification.

### 1.3.3 CNN-Based Classification Module

The third module is the **CNN-Based Classification Module**, which is the core intelligence of the system. This module loads a pre-trained Convolutional Neural Network model from the file isl_sign_language_cnn_fast.h5 using Keras. The CNN has been trained earlier on a large number of ISL gesture images so that it can recognize patterns in hand shapes, finger positions and overall gesture structure. The architecture consists of several convolutional and pooling layers followed by fully connected layers and a final Softmax output layer, which gives a probability distribution over all possible classes.

When a pre-processed image arrives from the previous module, it is passed through the CNN model using the model.predict() function. The model computes a set of probabilities, one for each class in the label set. The class with the highest probability is selected as the predicted sign. For example, if the highest probability corresponds to index 10 and the mapping dictionary index_to_class maps "10" to "A", then the system predicts that the gesture represents the letter "A". This module also calculates the confidence level by taking the maximum probability and converting it into a percentage. In this way, the CNN-Based Classification Module transforms numerical image features into a meaningful gesture label that can be understood by the user.

### 1.3.4 Application Interface Module

The fourth module is the **Application Interface Module**, which is responsible for interaction between the user and the recognition system. In your project, this module is implemented using

**Streamlit**, a Python-based web application framework. At startup, Streamlit configures the page with a title, icon and layout, and displays a clean user interface with headings, descriptions and instructions. A radio button allows the user to select between *Upload Image* and *Use Webcam* modes. Depending on the choice, the corresponding input component (file_uploader or camera_input) is displayed on the screen. Once the user provides an image, it is shown back in the interface as a preview so that the user can confirm the correctness of the captured gesture.

The interface also contains a button such as **"Predict Sign"** or **"Predict Sign"**. When this button is clicked, the application triggers the processing pipeline: it sends the current image to the pre-processing module, forwards the result to the CNN classification module and then collects the prediction output. Additional helper components in the sidebar explain how to use the system, give information about ISL recognition and describe the internal working of the model. Through this module, technical functions of the system are presented in a user-friendly manner, making it accessible to non-technical users as well.

### 1.3.5 Prediction and Result Visualization Module

The final module is the **Prediction and Result Visualization Module**, which presents the outcome of the recognition process in a clear and informative way. Once the CNN returns the predicted class index and probabilities, this module converts the index into a human-readable label using the index_to_class mapping loaded from the class_labels.json file. The predicted sign is then displayed prominently in the interface, often as a bold or highlighted text such as "Predicted Class: A". The confidence score, computed as the maximum probability multiplied by 100, is displayed as a percentage so that users can understand how sure the model is about its decision.

In addition to the main prediction, this module may also show the **hand region sent to the model**, so that users can visually see what part of the image was used for classification. An expandable section (for example, a Streamlit expander) can list the probabilities for all classes, allowing more detailed analysis of the model's behavior. This is particularly useful for debugging and for demonstrating the performance of the system in presentations or project reviews. By combining textual output, image visualization and probability information, the Prediction and Result Visualization Module completes the user experience and clearly demonstrates the effectiveness of the proposed ISL recognition system.

# CHAPTER II – LITERATURE REVIEW

### 1. INEGI (2021) – Disability Statistics and Communication Needs

This report published by INEGI provides national-level statistics on individuals with disabilities and highlights the communication challenges faced by populations with hearing impairments. The document outlines demographic trends, prevalence of disabilities, educational barriers, and social limitations experienced by people who rely on non-verbal communication methods such as sign language. These statistics serve as a crucial foundation for understanding the social importance of sign language recognition systems. The report emphasizes the necessity of creating inclusive technologies and tools that bridge communication gaps between disabled individuals and the rest of society. By presenting real-world challenges and inequalities, this reference reinforces the importance of developing AI-driven systems such as automated sign language recognition platforms that improve accessibility, communication independence, and inclusiveness.

### 2. Chollet, F. (2021) – Foundations of Deep Learning with Python

François Chollet's book, *Deep Learning with Python*, offers a comprehensive introduction to deep learning theory, neural network models, and practical implementation using the Keras framework. The author explains the mathematical intuition behind neural networks, the importance of convolutional architectures for image tasks, and best practices for training efficient and scalable models. This reference is foundational for any system using CNNs or LSTMs, as it provides both theoretical knowledge and practical coding strategies that guide model development. In the context of sign language recognition, Chollet's explanations of convolution layers, feature extraction, and backpropagation directly support the design of gesture recognition networks. This reference strengthens the methodological grounding of projects involving training and deploying neural networks for classification, object detection, and image processing applications.

### 3. Ma et al. (2022) – Large Margin LSTM Training for Language Models

This work presents an improved training strategy for Long Short-Term Memory (LSTM) neural networks through a large-margin objective function. The authors argue that typical training methods do not enforce sufficient separation between classes, leading to lower accuracy in

challenging sequence tasks. By introducing a margin-based constraint, the model becomes more robust and better at distinguishing subtle differences in sequential patterns. While the paper focuses on neural language modeling, the underlying LSTM advancements can be applied to gesture recognition, especially dynamic signing or continuous sign translation systems. The research highlights the value of improving temporal modeling capabilities, which is essential for future extensions of ISL or ASL recognition systems that require continuous gesture sequence interpretation.

**4. Dai et al. (2020) – Two-Stream Attention-Based LSTM for Action Recognition**

This study proposes a two-stream architecture combining spatial and temporal information processed through attention-enhanced LSTM layers for human action recognition. The model receives both RGB frames and optical flow images, enabling it to capture static posture and motion dynamics simultaneously. The attention mechanism improves the network's focus on relevant regions of the frame, reducing noise and improving accuracy. This approach is highly relevant to sign language recognition because gestures involve fine-grained movements that require both spatial and temporal analysis. The paper demonstrates how hybrid CNN–LSTM models can outperform single-stream networks, providing a valuable framework for designing advanced sign recognition systems that interpret continuous or dynamic gestures.

**5. Agarwal et al. (2021) – Deep Learning Framework for Visual-to-Caption Translation**

Agarwal and colleagues explore a deep learning architecture that converts visual inputs into descriptive captions using CNN and RNN models. The framework extracts high-level image features using convolution layers and then passes them through LSTM-based sequence generators to produce natural-language captions. Although the paper focuses on caption generation rather than gesture recognition, its methodology is closely related to sign language translation systems. The integration of CNN feature extraction with sequence modeling demonstrates how visual information can be transformed into text — a capability that future ISL recognition systems may adopt for end-to-end gesture-to-text translation. This work is important because it shows the potential of multimodal deep learning for bridging communication gaps.

**6. Vasani et al. (2020) – Indian Sign Language Generation Using GANs and Sentence Processing**

This study presents a deep learning framework for generating Indian Sign Language (ISL) gestures using **Generative Adversarial Networks (GANs)** combined with sentence processing techniques. The authors aim to automatically convert text-based sentences into sign language representations by learning the structural and grammatical properties of ISL. The use of GANs enables the creation of high-quality gesture frames that resemble real ISL signs, helping in situations where training data is limited. The paper emphasizes linguistic alignment between natural language and sign language structure, ensuring that generated gestures preserve semantic meaning. The proposed approach is particularly relevant for developing **bidirectional sign language communication systems**, where text or speech can be translated into sign gestures. This study contributes significantly by demonstrating that GAN-based models can support educational tools, virtual avatars, and assistive technologies for the deaf community.

**7. Jayadeep et al. (2020) – MUDRA: CNN-Based Indian Sign Language Translator for Banking Applications**

This work introduces **MUDRA**, an ISL translation system designed specifically for banking environments to enhance accessibility for hearing-impaired individuals. The system employs a Convolutional Neural Network (CNN) to classify static ISL hand gestures captured through a webcam. The paper highlights the importance of domain-specific applications of sign recognition, particularly in service sectors like banking where communication barriers frequently occur. The authors describe a user interface that enables customers to interact with banking services using intuitive sign gestures. The study demonstrates how CNN models offer high accuracy in recognizing static signs and can be integrated into real-world systems. This reference is valuable to your project because it shows a successful implementation of ISL recognition using CNNs, validating the effectiveness of deep learning for gesture classification tasks.

**8. Ru & Sebastian (2023) – Real-Time American Sign Language Interpretation System**

This paper proposes a real-time ASL interpretation framework that uses deep learning for instant gesture recognition. The authors focus on creating a system that performs efficiently under real-

world conditions, handling challenges such as varying lighting, diverse backgrounds, and hand orientation differences. The method integrates a gesture detection component with a classification model to deliver accurate interpretations on live video input. The emphasis on **real-time performance** is directly relevant to your system, which also uses webcam-based gesture capture through Streamlit. The research demonstrates the increasing trend toward practical deployment of sign recognition tools rather than purely academic models. The challenges and solutions presented in this paper align closely with your project's objectives of building a fast, accurate, and user-friendly ISL recognition system.

**9. Srinivasa et al. (2017) – Analysis of Facial Expressiveness in Reaction to Videos**

Although not directly related to hand gesture recognition, this study examines how facial expressions reflect emotional reactions while watching video content. The authors use computer vision techniques to track facial features and measure expressiveness. The relevance of this work lies in its demonstration of how visual cues, including subtle movements, can be analyzed using machine learning methods. In advanced sign language systems, facial expressions and body posture are essential components of sign semantics, especially in languages where facial gestures carry grammatical meaning. This study highlights the potential for integrating multimodal cues — hand gestures + facial expressions — in future sign language recognition frameworks. It thus contributes to understanding the broader landscape of visual communication analysis within AI systems.

**10. Rahman et al. (2022) – Continuous Sign Language Interpretation to Text Using Deep Learning**

This work explores an end-to-end deep learning model capable of converting continuous sign language gestures into textual representations. Unlike systems that focus only on isolated static signs, this research tackles dynamic, sequential signing, which is significantly more challenging. The authors use a combination of CNNs and recurrent neural networks (such as LSTMs or GRUs) to model both spatial and temporal features of sign gestures. The system is trained to identify sign boundaries, recognize gestures in sequence, and convert them into meaningful sentences. This study is particularly important because it illustrates the progression from single-gesture recognition to continuous sign interpretation — a direction in which your ISL system can evolve in the future.

The paper demonstrates promising accuracy and highlights the potential for automated sign-to-text translation systems.

**11. Cheng et al. (2021) – Real-Time Chinese Sign Language Recognition Using Pose Estimation and Attention Networks**

Cheng and colleagues propose a real-time recognition system for Chinese Sign Language that integrates **pose estimation** with an **attention-based deep learning network**. Their approach starts with extracting human body keypoints using a pose estimation model, which significantly reduces background noise and isolates the critical joints involved in signing. These pose-based features are then fed into an attention network that dynamically weighs the importance of different joints and frames, improving recognition accuracy. The real-time capability of this system demonstrates its potential for deployment in interactive applications and assistive communication tools. This study is highly relevant to modern gesture recognition research because it shows how combining pose estimation with attention mechanisms can reduce computational load while maintaining strong accuracy. The methodology supports the idea that sign language recognition can benefit from multimodal data and more intelligent feature-selection strategies.

**12. García-Bautista et al. (2021) – Mexican Sign Language Recognition Using RGB-D Information**

This research utilizes **RGB-D cameras**, which capture both color and depth information, to enhance Mexican Sign Language (LSM) word recognition. By incorporating depth data, the authors overcome common challenges such as background clutter, lighting variability, and hand-over-face occlusions. The system extracts spatial-temporal features from both RGB and depth streams and uses machine learning classifiers to recognize sign language words. One of the major contributions of this paper is demonstrating that depth information significantly improves accuracy in distinguishing similar hand shapes and movements. This study broadens the understanding of multimodal input for sign language systems and highlights the potential advantages of using low-cost depth sensors like Kinect. Although your project uses only RGB input, this research shows how richer input data can benefit future expansions.

**13. Ortiz-Farfán & Camargo-Mendoza (2020) – Computational Model for Sign Language**

**Recognition in the Colombian Context**

This study presents a computational model tailored specifically for recognizing Colombian Sign Language by leveraging computer vision and machine learning techniques. The authors emphasize the cultural and linguistic uniqueness of different sign languages and the necessity of developing region-specific models. Their system focuses on extracting meaningful static hand gesture features and classifying them using supervised learning algorithms. The research contributes valuable insights into dataset creation, sign language structure, and annotation challenges faced during model development. The relevance of this work lies in its reinforcement that gesture recognition systems must be adapted to the linguistic rules, hand configurations, and expression styles of each sign language. For your project, this highlights the importance of designing models specifically for **Indian Sign Language (ISL)** rather than adapting foreign-language datasets.

**14. Natarajan et al. (2022) – End-to-End Deep Learning Framework for Sign Language Recognition and Translation**

Natarajan and coauthors propose an end-to-end deep learning system that not only recognizes sign gestures but also translates them into natural language sentences and generates sign language videos. The framework includes gesture recognition, semantic translation, and video synthesis modules, integrating multiple deep learning technologies such as CNNs, RNNs, sequence encoders, and GAN-based generators. The significance of this work lies in its comprehensive nature — moving from isolated gesture recognition to full sign language understanding and multimodal generation. This study demonstrates future directions for sign language technology, aiming to bridge the communication gap through bidirectional conversion between text and sign. The methodology strengthens the concept of applying deep learning not just to classify gestures but also to interpret and generate linguistic meaning at a higher level.

**15. Wang et al. (2022) – SignGest: Sign Language Recognition Using Acoustic Signals on Smartphones**

This innovative study introduces a new modality for sign language recognition by using **acoustic signals generated from smartphone speakers and microphones**. Instead of relying solely on vision-based systems, the authors utilize ultrasonic echoes to detect hand shapes and movements.

The approach demonstrates that sign language gestures can be recognized through signal reflections, allowing operation even in low-light or visually noisy environments. The system processes echo patterns to extract gesture features and classifies them using machine learning models. Although this method differs significantly from camera-based recognition, it offers valuable insights into alternative sensing technologies. For your project, this reference broadens the scope of understanding by showing that gesture recognition does not have to depend solely on image data — multimodal sensing can enhance recognition robustness in challenging conditions.

# CHAPTER III – Existing System

## Sign Language Interpreting System Using Recursive Neural Networks

### 1. Introduction

According to the National Institute of Statistics and Geography of Mexico (INEGI) [1], there are around 1,350,802 people with hearing disabilities in Mexico. As is known, these people communicate with each other using sign language fluently; however, most people whouse speech are unfamiliar with Mexican Sign Language (MSL), so communicating between them can be difficult when there is no interpreter or shared means of communication, such as a written language that both parties are proficient in.

On the other hand, artificial intelligence has advanced to rival human decisions in some areas [2]; such is the case of the discipline called deep learning. Within deep learning is the Recurrent Neural Networks or RNN, and part of this technique is the Long Short Term Memory or LSTM networks. These machine learning techniques are used to process.

Data sequences widely applied to research on automatic speech recognition and natural language processing [3]. For example, Ref. [4] proposed a neural network focused on human action recognition from videos. The authors utilized an attention mechanism to focus on the most important features of each frame. The authors propose a new neural network architecture to capture this temporal information. The network combines two streams:

• An LSTM stream to analyze the sequence of frames in the video.

• ACLSTM (Convolutional LSTM) stream captures spatial and transient information.

As a result, the method is more accurate than previously existing methods. On the other hand, Ref. [5] created a model for generating video descriptions. The model uses recurrent neural networks, specifically LSTMs, to capture the transient struc structureture of videos and generate descriptions. The model was trained using a dataset made of YouTubevideos; author authors claim their model outperforms previous models regarding sclaim their model out performs previous models regarding accuracy.

From a different perspective, Ref. [6] proposed a method to generate Indian Sign Language (ISL) videos from sentences using sentence preprocessing and generative adversarial network networks (GAN). Their method focuses on converting sentences into glosses (a present signs using written language) and generating synthetic video frames for each gloss using a GAN architecture. The generated videos use animated avatars and 2D/3Dgraphics. Despite the difficulties related to this research field, it is essential to highlight the importance of communication since it is fundamental to connecting people and communities; this motivates our work since, for the deaf community in Mexico, MSL is more thanameans of communication, it is a pillar of both their culture and their identity.

Despite the progress regarding public awareness of the importance of MSL, this type of communication still faces significant challenges in achieving its adoption by our society. For this reason, this work is more than just a technological exercise; it is a concrete manifestation of the commitment to including deaf people in our society. We believe that this can be reached by empowering the deaf community with technological tools that facilitate their communication, so we expect that this work contributes to achieving a more inclusive and equitable society. The contributions of this paper are:

• We designed four multi modal models based on spatial tracking, gesture recognition, and an RNN(Section 3).

• We tuned the proposed systems and identified the most suited model for the target application (Section 5).

• We tested the proposed system both in an offline and online scenario (Section 5).

• We evaluated the accuracy of the system in the offline and online scenarios (Section 5).

We expect our work to impact different social areas, such as education, the health care sector, transportation, finances, and daily life in general, helping deaf people interact with the rest of the people in equal circumstances. The remainder of this paper is organized as follows: in Section 2, we discuss related works to our proposed model; Section 3 describes the operation of the system we propose; Wedetail the materials and the creation process of our model, as well as the characteristics of the data set and the evaluations carried out on the model in Section 4; we present the results obtained in Section 5; next, our conclusions are presented in Section 6; finally,

we have the references and the Appendixes A and B.

## 2. Related Works

The research detailed in [7] focused on developing a sign language detection system to facilitate communication between the deaf community and banks. The main difficulty lies in the complexity of Indian Sign Language (ISL) gestures related to the banking domain. The lack of standardized datasets and the variability in gesture length were significant challenges. To address this, the proposed approach combines feature extraction using a Convolutional Neural Network (CNN) with an LSTM Recurrent Neural Network. This system was trained and tested on a customized ISL dataset.

Onthe other hand, ref. [8] focused on developing an American Sign Language (ASL) recognition system based on computer vision and deep learning. It also presents a publicly available motion-based ASL dataset for future research which contains thirty phrases ob tained from the book American Sign Languagefor Dummiesalongwithphrasesconsidered basic for communication. They consider three types of algorithms: ConvLSTM, Convo lutional Neural Network (CNN) and LSTM, and Mediapipe and LSTM, and compared them, with two main parameters to observe: the number of sentences in the model and the precision of the sentences in each one.

They used the the MediaPipe (an open source framework by Google useful for tasks such as face detection, hand tracking, and more) and LSTMmodel. They concluded with 97% accuracy in training and 79% accuracy within the tests carried out. Finally, they propose to use the Transformer algorithm to improve the performance of the system for future work. In the field of video interpretation, the authors of [9] implemented a methodology to evaluate expressiveness in facial videos captured of people responding to ads based on deep learning; specifically, they used an LSTM network. They detailed the structure of the LSTMmodelwhich includes a dropout layer which turns off neurons randomly selected to avoid overfitting the NN. The model was trained to determine expressiveness based on facial cues, and user interfaces were provided through a website with a camera connected to an IoT board to display the results.

In turn, Ref. [10] proposed a sign language-to-text translation system using deep learning models, specifically CNN and LSTM. The system was trained on an enriched dataset that contains common phrases used in professional meetings. The results show that the LSTM model outperforms the

CNN model. The authors of [11] proposed a model for video recognition using computer vision techniques for Chinese sign language which achieves an accuracy of 85%. They designed a model based on an LSTMneural network to capture skeletal tracking information.

They ended the paper by suggesting further improvements to their model's accuracy by retrain ing using an expanded dataset and incorporating facial expression recognition. Also, in the research detailed in [12], a system focused ontranslating MSLusingRGB-D information was introduced. The authors built a semantic corpus of 53 words belonging to Mexican Sign Language (MSL) from scratch. From this corpus, they captured the trajectory of hand movements using a Kinect sensor; then, a KNN-based algorithm eliminated inconsistencies and noise in the extracted patterns. They used a Multilayer Perceptron Neural Network for word classification and implemented the Backpropagation algorithm for training.

Finally, they validated their system using the K-Fold Cross Validation method, obtaining an average accuracy percentage of 93.46%. Ortiz [13] created a database containing Colombian Sign Language (CSL) signs and used it to train a CNN to recognize CSL gestures. They found several activation functions failed to respond adequately; however, Dropout improved the model's accuracy and loss. Additionally, they observed that the number of layers is unrelated to the model's accuracy. Natarajan [14] proposed deep learning-based techniques to enhance sign language video recognition, translation, and generation.

The researchers utilized the MediaPipe library, a hybrid CNN, andabidirectional long short-term memory(BLSTM)modelforpose detail extraction and text generation. They utilized a hybrid machine translations neural network combined with MediaPipe and a dynamic generative adversarial network (GAN) to generate sign language gestures. The proposed model achieved over 95% classification accuracy and significantly improved visual quality. The dynamic GAN was evaluated using different multilingual datasets, and excellent results were produced. In contrast, Ref. [15] presented a sign language recognition system called SignGest that uses acoustic signals from a smartphone to capture sign language gestures and a CNN model to distinguish between different gestures.

In addition, it used a deep convolutional GANtogenerate training data. They implemented the system on a server and an Android. smartphone without additional hardware. The experimental results showed that SignGest achieved its target performance which they assessed using the Structural Similarity Index Measure (SSIM). Additionally, in [16], a method for recognizing words

in Japanese Sign Language (JSL) based on the abstraction of hand signals was implemented; this was achieved using measuring the hand shape, position, and movement. The proposed system uses a contour based method and template matching for hand shape recognition. They also explored a second method based on the Hidden Markov Model.

The system was evaluated using a dataset of 400 JSL words, and results showed a recognition rate of 33.8%. In turn, Ref. [17] introduced an interactive Taiwanese sign language learning system based on Microsoft Kinect. They processed the color image, depth image, and skeleton points of the hands and body to extract relevant information. Then, they fitted an support ing vector machine (SVM) classification to identify the intended sign language word, and the experimental results showed a recognition rate of 90.2% and hand motion detection of 98%. In the paper [18], the authors presented research focused on the automatic recognition of Mexican Sign Language (MSL) by analyzing body movement and facial gestures.

The proposed system employs multiple gestures, including hands, body, and face, using a dataset of 3000 samples of 30 different MSL signs. They used a depth camera to capture the coordinates of the movements while utilizing RNN for classification. The results of their study demonstrated a high accuracy of the system, reaching 97% on clean test data and 90% ondata with a high level of noise, highlighting the robustness and effectiveness of the proposed approach. Another relevant work in MSL recognition is [19], the authors presented a system de veloped to recognize static signs of the dactylological alphabet.

They utilized the MediaPipe framework andaCNNmodeltointerprettheletters representing the hand signals captured by a camera. A database composed of 336 test images was used, their system achieved an accuracy of 83.63% with the CNN model. In addition, they evaluated samples of each letter and achieved an accuracy of 84.57%, a sensitivity of 83.33%, and a specificity of 99.17%. The advantage of their proposal lies in its implementation capacity on low-consumption equipment, allowing real-time classification and improving the accessibility and usability of the system. The authors of [20] proposed a method for the recognition of the static alphabet of the MSLcapable of learning the shape of each of the 21 letters from examples.

The method they proposed first normalizes each example of the training set into a 3D pose through utilizing the principal component analysis (PCA). The input data was captured using a 3D sensor, andthemethodgenerates three types of features representing each shape. The authors used a set of data acquired in a laboratory, their approach achieved 100% accuracy. In addition, the features

used in this method have a clear and intuitive geometric interpretation. Another interesting work is [21], in this work, the authors explored a new lexical video dataset of MSL, which includes the most frequently used dynamic gestures.

Each gesture class was composed of different versions of videos recorded under uncontrolled conditions. This dataset, named MX-ITESO-100, contains a lexicon of 100 gestures and 5000 videos made by three participants, showing various grammatical elements. Furthermore, they evaluated the dataset using a two-step neural network model, reaching an accuracy of 99%, making it a useful benchmark for training future machine learning models in computer vision systems. Another work related to project we propose is [22]. The authors designed a bidi rectional sign language translation system to bridge the communication gap between hearing and deaf people. They evaluated Several deep learning models, including RNN, bidirectional RNN (BRNN), LSTM, GRU, and Transformers to identify the most accurate sign language recognition and translation model. Also, they used a Keypoint (the spatial locations of important landmarks in the image) detection approach through MediaPipe to track and understand gestures.

The system incorporates an intuitive graphical interface. which allows translation between MSL and Spanish in both directions, facilitating the entry of signs or text and obtaining the corresponding translations. The evaluation results demonstrated high accuracy, with the BRNN model standing out with an accuracy of 98.8%.

The authors underlined the importance of hand characteristics in sign language recognition and offered a promising solution to improve communicative accessibility and promote the inclusion of people with hearing disabilities. In [23], the authors presented a model for sign language that combines spatial and temporal attention with a general neural network. The model they designed uses self attention mechanisms to extract relevant information from node and edge features in both spatial and temporal domains.

The architecture of their system is divided into three branches: a graph-based spatial branch that identifies spatial dependencies, a graph-based temporal branch the temporal branch that reinforces temporal dependencies in sequential hand skeleton data, and a general neural network branch that improves the generalization capability of the model increasing its robustness. They evaluated the

system performance on the MSL, the Pakistani Sign Language (PSL) datasets, and the American Sign Language Large Video Dataset (ASLLVD), including 3D joint coordinates for face, body, and hands; the authors conducted experiments on both individual gestures and their combinations. The model achieved a 99.96% accuracy on the MSL dataset, 92.00% on PSL, and 26.00% on the ASLLVD dataset, covering over 2700 classes.

On the other hand, the research presented in [24] focused on identifying lexical signs. This study proposed a method to identify phonetic units in MSL videos using an unsu pervised model, to achieve this, they used an image thresholding approach based on the Motion-Retention Model of Liddell and Johnson. The goal was to identify the smallest linguistic units containing relevant information, avoiding the loss of sub lexical data that might go unnoticed by most natural language processing (NLP) algorithms.

Furthermore, the method allows for removing noisy or redundant video frames, thus reducing computa tional costs. The authors tested their algorithm tested on a collection of MSL videos, and human judges assessed the relevance of the extracted segments. The results indicated that the frames selected by the algorithm contained enough information to be comprehensible for signers. In some cases, up to 80% of available frames could be discarded without loss of comprehensibility, which has direct implications for future electronic representation, transmission, and processing of sign language. The authors of [25] introduced an interactive platform for teaching MSL based on artificial intelligence (AI) models, which they called DAKTILOS. This platform allows the native integration of real-time tracking of hand movements from 2D images. DAKTILOS recognizes and evaluates both static and dynamic signs of the LSM alphabet the user signals.

First, the system identifies 21 critical points of the hand using the MediaPipe model; next, these are dynamically compared to the target hand configurations (letters) using the FingerPose classifier. Finally, the system computes a scores and determines whether the sign was performed correctly, providing visual feedback. The platform allows for exploring 27 letters, showing the correct configuration with a 3D hand.

Preliminary tests were conducted with eight subjects to evaluate the functionality of DAKTILOS.

These works support the idea of using RNN to generate a common means of com munication to facilitate communication between people with hearing disabilities who use sign language, with people who can hear; however, despite previous efforts, the following issues remain unaddressed:

• Previous systems depend on the specific signs of the language, that is to say, they are language-oriented and as a consequence, there are particular design constraints.

• The accuracy of most of those approaches is still low for practical scenarios.

• Theauthorshavelimitedtheexperimentstheyconductedontheirschemestooffline tests.

3. The Proposed System The main idea is to utilize spatial coordinates of the body parts involved in MSL, specifically the hands, torso, and face because gestures play an important role in MSL.

# CHAPTER IV – PROPOSED SYSTEM

## 4.1 Introduction

The proposed system is an intelligent **Indian Sign Language (ISL) Gesture Recognition System** designed to classify static hand gestures in real time using a combination of **Convolutional Neural Networks (CNN)** and **MediaPipe-based hand detection**. The system aims to bridge the communication gap between hearing-impaired individuals and the general population by enabling automatic interpretation of ISL gestures from images or webcam input.

Unlike traditional gesture recognition systems that depend heavily on background conditions, fixed lighting, or handcrafted features, the proposed system uses **deep learning**, advanced **image pre-processing**, and a **Streamlit-based user interface** for smooth interaction.

The overall architecture consists of several integrated modules:

1. Hand detection using MediaPipe
2. Image pre-processing
3. CNN model classification
4. Streamlit-based application interface
5. Real-time prediction and visualization

This chapter details the architecture, algorithms, workflow, and functional components of the proposed system.

### 4.2 System Architecture

The proposed system follows a modular architecture that processes user input through different stages before generating the final recognition result.

**Key Components:**

1. **Input Acquisition Module** (image upload or webcam)
2. **Hand Detection & Cropping Module** (MediaPipe Hands)
3. **Pre-processing Module** (resize, normalize)
4. **Feature Extraction & Classification Module** (CNN model)

5. **Prediction Display Module** (Streamlit)

**Workflow Summary:**

- The user provides an image or captures a gesture using webcam →
- MediaPipe detects the hand region and extracts landmarks →
- The detected hand is cropped and standardized →
- The CNN model predicts the gesture class →
- The system displays the predicted label with confidence.

This pipeline ensures speed, accuracy, and robustness across different environments.

### 4.3 Working Mechanism of the Proposed System

#### 4.3.1 Hand Detection Using MediaPipe

MediaPipe Hands is employed to detect the hand in real time. It identifies **21 hand landmarks**, calculates bounding box coordinates, and extracts the hand region from the input image.

Steps:

1. Convert the input image to RGB format.
2. Run MediaPipe Hands to detect hand landmarks.
3. Compute the minimum and maximum landmark points.
4. Crop the bounding box with a margin to avoid cutting important regions.
5. Use fallback center-cropping if no hand is detected.

This ensures consistent and high-quality input for the classification model.

#### 4.3.2 Pre-processing Module

The cropped hand image undergoes pre-processing to match the CNN training specifications:

1. **Resizing** – The image is resized to 64 × 64, the expected input dimension of the CNN model.
2. **Normalization** – Pixel values are scaled from [0–255] to [0–1].

3. **Channel Adjustment** – Ensures all images are in RGB format.
4. **Batch Expansion** – Adds extra dimension for model inference.

Pre-processing ensures high-quality input and reduces model prediction errors.

### 4.3.3 CNN-Based Classification Model

The classification model used is **isl_sign_language_cnn_fast.h5**, which was trained on ISL gesture images. The CNN model extracts spatial features such as:

- Contours
- Finger positions
- Edges
- Palm orientation
- Hand shape patterns

The final layer uses **softmax activation** to generate probability values for each class. The predicted class corresponds to the index with the highest probability. The mapping from index to class label comes from class_labels.json.

CNN advantages:

- Automatically extracts features
- High accuracy for image-based tasks
- Robust against noise
- Fast inference

## 4.4 Functional Modules of the Proposed System

### 4.4.1 Input Module (Upload / Webcam)

The system offers two modes:

**Upload Image Mode**

- User uploads JPEG/PNG image.

- Image is displayed for confirmation.

**Webcam Capture Mode**

- User takes a picture using the laptop camera.
- Ideal for real-time gesture recognition.

Both modes generate an RGB image which is forwarded to MediaPipe for hand extraction.

### 4.4.2 Hand Processing Module

This module isolates the **Region of Interest (ROI)**—the hand.

Functions:

- Detect hand landmarks
- Compute bounding region
- Crop hand area
- Return cropped image to pre-processing stage

This ensures only relevant gesture information is passed forward.

### 4.4.3 Classification Module

Handles gesture recognition using the CNN model.

Steps:

1. Load CNN model with load_model()
2. Load class labels
3. Predict using model.predict()
4. Extract highest probability
5. Map predicted index to class label

Outputs:

- Predicted sign

- Confidence score in %

### 4.4.4 Output Visualization Module

Responsible for presenting results:

- Displays the hand region used for prediction
- Shows predicted class label
- Shows probability/accuracy
- Displays class-wise probability distribution (optional expander)

This module enhances user understanding and trust in the system.

### 4.5 Algorithm of the Proposed System

**Algorithm: ISL Gesture Recognition**

**Input:** Image or Webcam frame

**Output:** Recognized ISL character / digit

1. Start
2. Read input image
3. Convert image to RGB
4. Use MediaPipe to detect hand landmarks
5. If hand detected:

   Crop hand region

   Else:

   Perform fallback cropping
6. Resize image to 64×64
7. Normalize pixel values
8. Expand dimensions for CNN
9. Load pretrained CNN model
10. Run prediction → get probabilities
11. Select class with maximum probability

12. Display predicted gesture and confidence
13. End

## 4.6 Advantages of the Proposed System

- **Real-time recognition** using webcam stream
- **High accuracy** due to CNN feature extraction
- **Improved reliability** with MediaPipe hand detection
- **User-friendly interface** powered by Streamlit
- **Lightweight and fast**, suitable for laptops and low-end hardware
- **Modular architecture** allows easy improvements and extension

## 4.7 Summary

The proposed system integrates AI, deep learning, and computer vision to deliver an intelligent solution for ISL sign recognition. By leveraging **MediaPipe hand detection**, **CNN classification**, and an easy-to-use **Streamlit interface**, the system provides fast, accurate, and practical sign recognition capabilities. This architecture serves as a strong foundation for future developments such as dynamic gesture recognition, real-time continuous sign translation, and multilingual sign language support.

## 4.1 UML Diagrams

To clearly represent the structure and behavior of the proposed Indian Sign Language (ISL) Recognition System, several **Unified Modeling Language (UML)** diagrams are used. These diagrams help in understanding the interaction between the user and the system, the internal classes/modules, the flow of operations and the overall control logic. The main UML diagrams considered in this project are:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram

Each of these diagrams is described in detail below.

### 4.1.1 Use Case Diagram

The **Use Case Diagram** represents the interaction between the user and the ISL Recognition System at a high level. It shows what major functions the system provides and how the user can access them. In your project, the main actor is a single **User** who wants to recognize an ISL hand sign using either an uploaded image or a webcam capture. The system is responsible for taking this input, processing it and returning a prediction.

From the user's point of view, the system offers several main use cases: launching the application, choosing an input mode, uploading or capturing a gesture image, asking the system to predict the sign, and viewing the recognition result along with confidence. Optional use cases such as viewing class-wise probabilities provide more detailed feedback useful for understanding and evaluation. These use cases summarize the external behavior of the system without exposing internal implementation.

This diagram is important in documentation because it clearly defines the **functional requirements** of the system in a user-centric way. It helps reviewers and non-technical stakeholders quickly understand what the system does, who uses it and how they interact with it. Internally, each use case maps to multiple modules such as image handling, hand detection, preprocessing and classification.

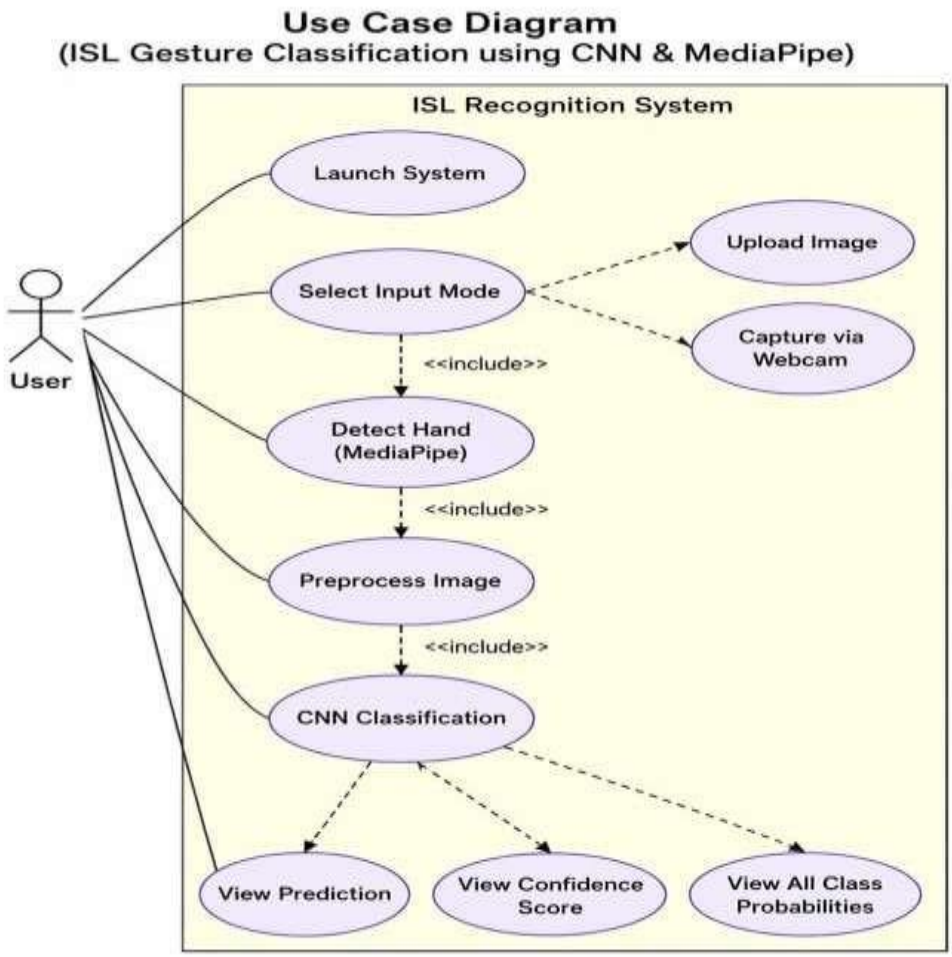



**Figure 5: Use Case Diagram**

### 4.1.2 Class Diagram

The **Class Diagram** models the internal structure of the proposed system in terms of logical classes, their attributes, methods and relationships. Although your implementation is in Python using functions and modules, it can still be conceptualized as a set of collaborating classes. This abstraction helps explain responsibilities and separation of concerns within the system.

At the center is the ISLApp class, which coordinates user interaction through the Streamlit interface. It makes use of helper classes like ImageHandler for managing uploaded or captured

images, HandDetector for MediaPipe-based hand cropping, Preprocessor for resizing and normalizing images, CNNModel for loading and running the trained model, and LabelManager for mapping indices to sign labels. Finally, PredictionResult stores the model output in a structured form, including predicted class and probabilities. The relationships show how these components depend on each other. ISLApp aggregates or composes the other classes, invoking their methods to complete the recognition pipeline. This design supports modularity: for example, changing the CNN model or replacing MediaPipe with another detector would only require changes in their specific class, not in the entire system.

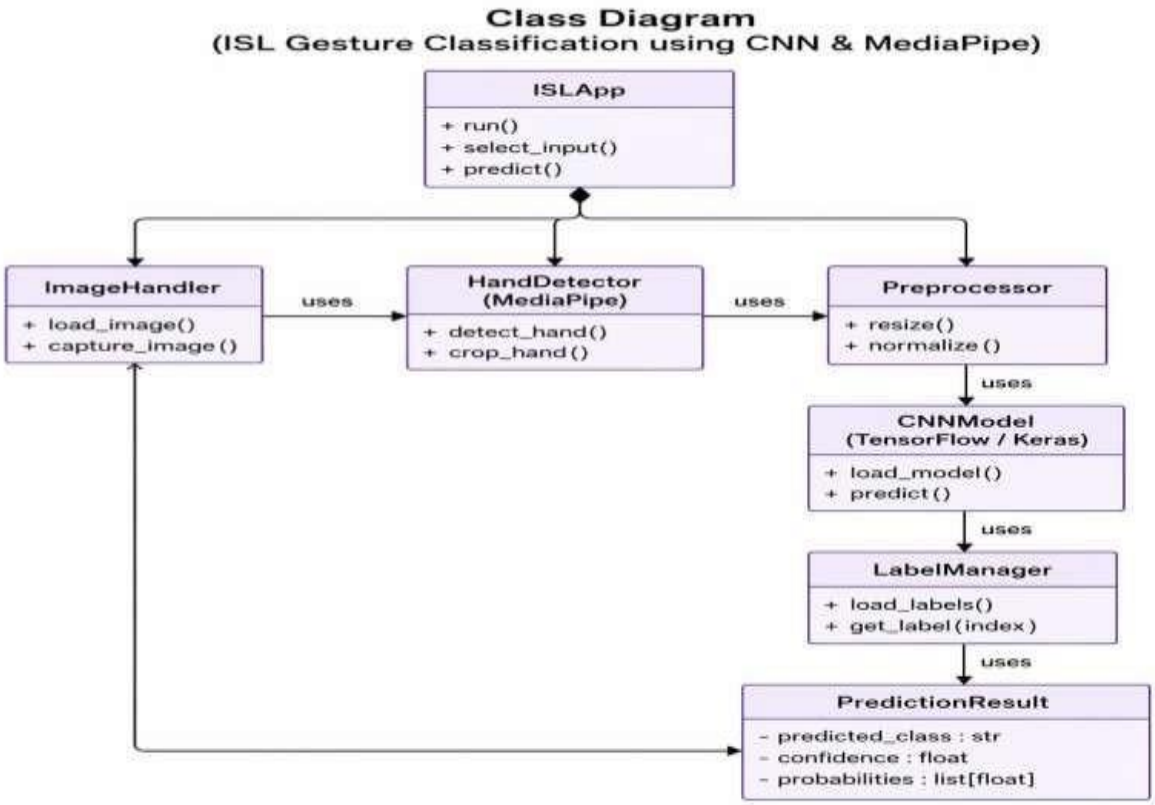



**Figure 6: Class Diagram**

### 4.1.3 Sequence Diagram

The **Sequence Diagram** explains how the different objects of the system interact over time for a specific scenario. In your project, one of the most important scenarios is: *"User captures an ISL*

*gesture using webcam and gets the predicted sign."* This diagram shows the flow of messages between the user interface and backend components in order. The sequence starts when the user chooses "Use Webcam" and captures an image. The ISLApp receives this event and calls ImageHandler to load and convert the camera image. Next, ISLApp invokes HandDetector to crop the hand region using MediaPipe. The cropped hand image is then passed to the Preprocessor, which resizes and normalizes it to meet the CNN's input requirements.

After preprocessing, the CNNModel object receives the image array and runs predict() to generate class probabilities. The predicted index is forwarded to LabelManager, which returns the corresponding ISL label. Finally, a PredictionResult object is created and sent back to ISLApp, which displays the prediction and confidence to the user via Streamlit. This diagram clearly demonstrates the **runtime collaboration** of system components.

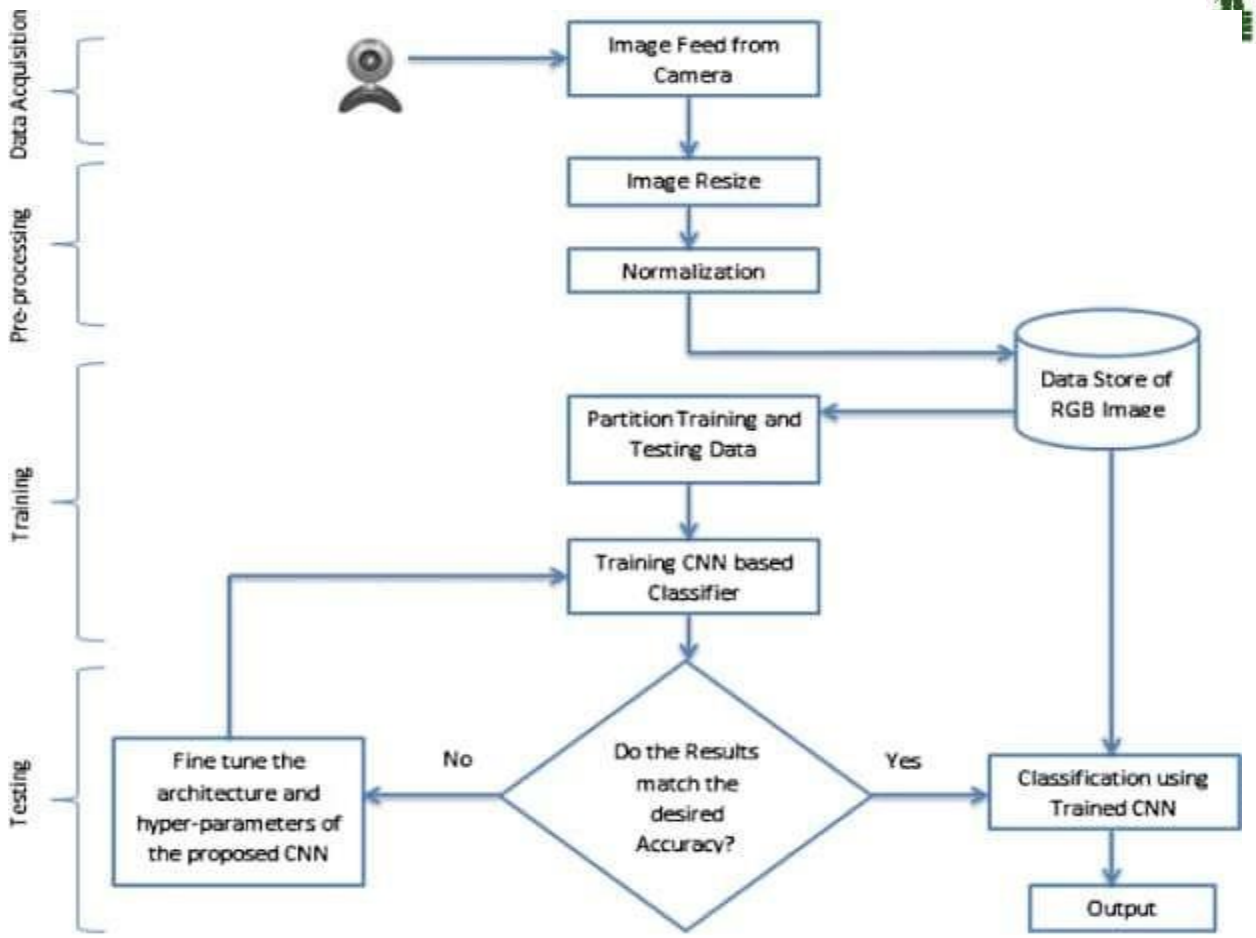



**Figure 7: Sequence Diagram**

### 4.1.4 Activity Diagram

The **Activity Diagram** focuses on the workflow or control flow of the ISL recognition process. It describes the different activities performed by the system from start to end when a user wants to recognize a gesture. Unlike the sequence diagram, which emphasizes object interaction, the activity diagram emphasizes **process steps and decision points**.

The process begins with launching the Streamlit application and choosing an input mode. Depending on the user's choice, the system either waits for an uploaded image or a webcam capture. After obtaining an image, the next activity is hand detection using MediaPipe. A decision node checks if a hand is detected; if not, a fallback cropping or an error message may be triggered. If detection is successful, the cropped hand is preprocessed—resized, normalized and reshaped for the CNN. The next activity is classification, where the preprocessed image is fed to the CNN

model. After prediction, the system converts the highest probability index into an ISL label and then displays the result. The flow ends after presenting the prediction, but the system waits for the next input, making the loop continuous. This diagram helps in understanding the overall **control logic** and is very useful for implementation and debugging.

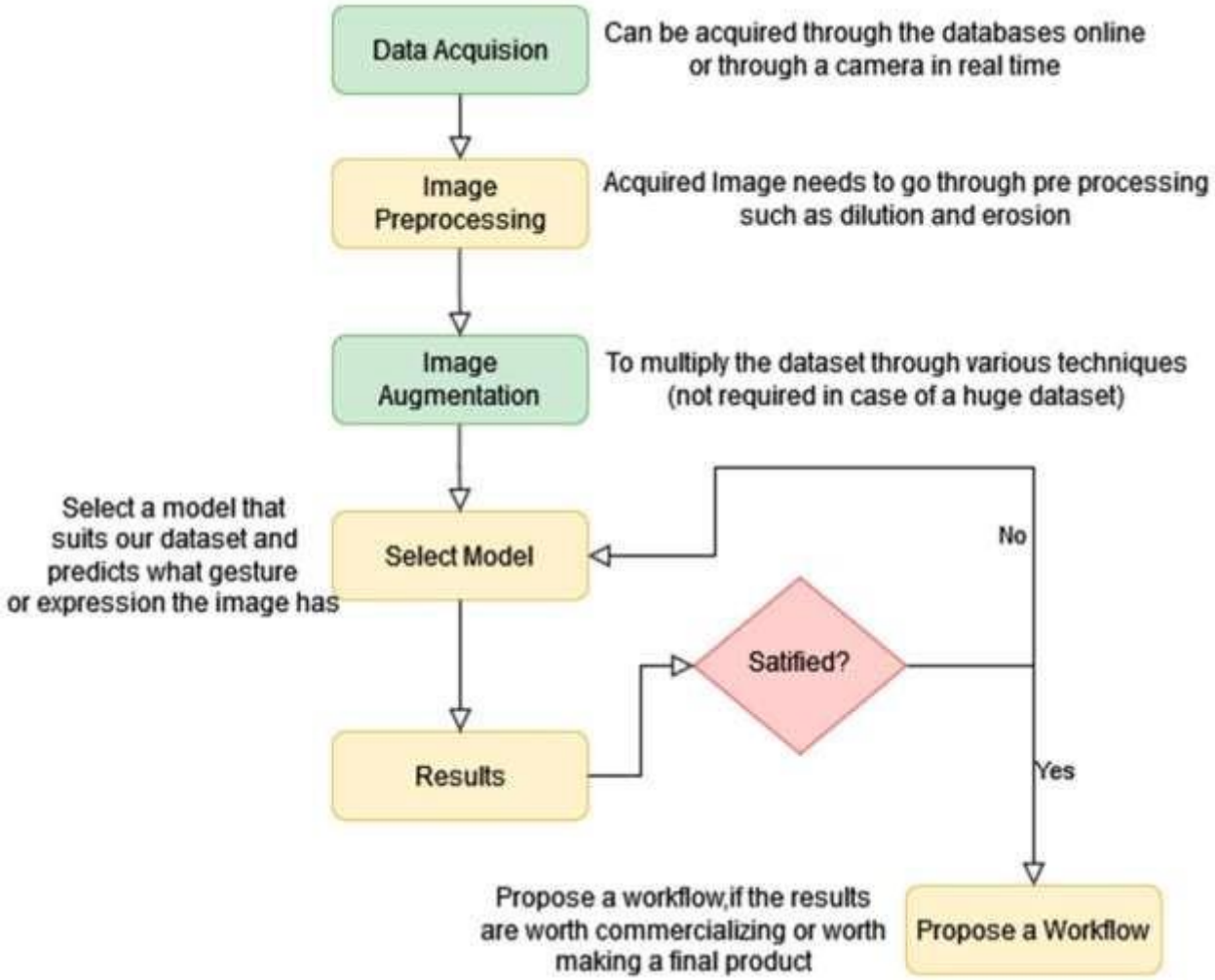



**Figure 8: Activity Diagram**

### 4.1.5 Component Diagram

The **Component Diagram** describes the high-level software components (subsystems) of your application and how they are interconnected. This is especially useful when you want to explain **deployment structure or modular design** without going into class-level details. In your ISL project, the main components can be grouped as UI, Processing, and Model components.

The **User Interface Component** (Streamlit App) is responsible for interaction with the user, capturing inputs and showing outputs. It communicates with the **Gesture Processing Component**, which internally uses MediaPipe for hand detection and a preprocessing pipeline for image standardization. These processed images are then forwarded to the **Model Inference Component**, which contains the trained CNN model and label mapping logic. Optionally, a **File Storage Component** (containing .h5 model and class_labels.json) is also involved.

This diagram makes it easier to see how different logical parts of the system are separated and how they depend on each other. For example, if you change the model file or retrain the CNN, you mainly affect the Model Inference Component, while leaving the UI and processing logic mostly untouched.

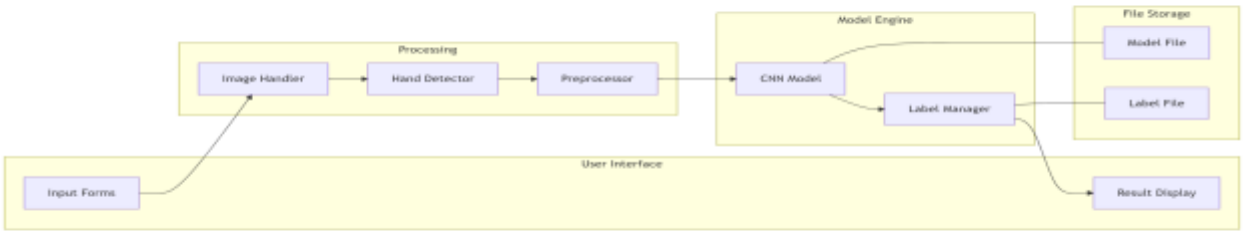



**Figure 9: Component Diagram**

## 4.1.6 Deployment Diagram (Logical View)

The **Deployment Diagram** shows how the system is physically or logically deployed across hardware or runtime environments. Even if your project runs on a single machine, it is still useful to illustrate how components map onto devices like a **Client Machine**, **Local Server**, and optional **Cloud Host**.

In a typical setup, the **Client Node** is a laptop or PC running a browser that displays the Streamlit application. The **Application Node** hosts the Python environment, Streamlit server, MediaPipe, OpenCV and the CNN model. If you later deploy this online, the Application Node could be in the cloud (e.g., on a cloud VM), while the client connects remotely over HTTP. The deployment diagram also shows that the model and label files are stored on the server file system and loaded at runtime.

This diagram helps convey that while the system appears simple to the end user, it relies on multiple layers: hardware (CPU/GPU), Python runtime, ML libraries and trained models. It is particularly useful when explaining scalability, performance considerations and future migration to cloud-based hosting.

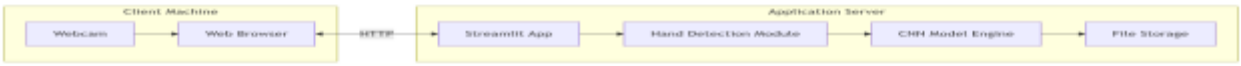



**Figure 10: Deployment Diagram**

## 4.2 Algorithms Used in the Project

The proposed Indian Sign Language (ISL) Recognition System utilizes several algorithms belonging to the domains of computer vision, deep learning, and geometric hand detection. These algorithms work together in different stages of the pipeline to ensure accurate, fast, and robust gesture recognition. This section explains the key algorithms used in the project, including their role, functioning, and relevance to sign language recognition.

### 4.2.1 MediaPipe Hand Detection Algorithm

The first core algorithm used in the system is **MediaPipe Hands**, a machine-learning-based pipeline developed by Google for high-fidelity hand detection and tracking.

**Working Principle**

MediaPipe Hands combines:

1. **Palm Detection Model** – finds the approximate location of the hand using a lightweight CNN.
2. **Hand Landmark Model** – predicts **21 key landmark points** on the hand, including fingertips, joints, and wrist.
3. **Bounding Box Estimation** – uses the landmarks to calculate a precise region of interest (ROI) for cropping the hand.

**Why It Is Used**

- Works **in real-time** with minimal computational power.
- Efficient even under **complex backgrounds** or partial occlusions.
- Removes irrelevant image regions, improving classification accuracy.

MediaPipe's reliability in detecting hand shapes makes it an ideal pre-processing step beforZgesture classification.

## 4.2.2 Image Preprocessing Algorithms

Preprocessing algorithms standardize the input image so the CNN model can interpret it consistently:

**(a) Image Resizing**

- Converts the hand-region image to a fixed resolution (e.g., **64×64** pixels).
- Ensures that the CNN receives uniform-sized images.

**(b) Normalization**

- Pixel values are scaled from **0–255** to **0–1**.
- Helps stabilize training and prediction by reducing numerical variations.

**(c) Cropping and Region Extraction**

- Uses MediaPipe landmarks to isolate the hand from the background.
- Significantly reduces noise and irrelevant visual information.

These preprocessing algorithms help improve recognition accuracy and reduce computational load.

### 4.2.3 Convolutional Neural Network (CNN) Algorithm

The main recognition engine of the system is a **Convolutional Neural Network (CNN)** trained on ISL gesture images.

**Working Principle**

A CNN consists of multiple layers:

1. **Convolutional Layers**
    a. Extract spatial features such as edges, lines, curves, and texture patterns.
    b. Learn the shape of each sign, including finger positions and palm geometry.
2. **Pooling Layers**
    a. Reduce feature dimensionality
    b. Strengthen the model's ability to detect features even when the hand is slightly shifted.
3. **Fully Connected (Dense) Layers**
    a. Combine learned features to form a classification decision.
4. **Softmax Output Layer**
    a. Converts raw scores into probabilities for each sign class.

**Why CNN Is Used**

- Automatically learns important features (no manual feature engineering required).
- High accuracy for image-based recognition tasks.
- Robust against variations in lighting and background.

The CNN model used in the project (isl_sign_language_cnn_fast.h5) has been fine-tuned to detect ISL gestures efficiently, making it the core decision-making algorithm.

### 4.2.4 Softmax Classification Algorithm

Once the CNN extracts features, the **Softmax classifier** is used to determine the most probable gesture class.

### Working Principle

The Softmax function converts the output vector into a probability distribution:

Where:

- *ziz_i*zi = score for class *i*
- *nn*n = number of gesture classes

**Purpose**

- Ensures the model outputs probabilities summing to 1
- Helps in selecting the gesture with the **highest probability**
- Used for confidence measurement in predictions (e.g., 93.45%)

Softmax is crucial for making the output readable and interpretable for users.

### 4.2.5 Argmax Decision Algorithm

After calculating class probabilities, the **Argmax algorithm** is used to identify the predicted class.

**Predicted Class=argmax(Softmax Output)Predicted**

**Why It Is Used**

- Simple and efficient
- Selects the class with maximum confidence
- Used in real-time prediction due to its low computational cost

It helps map the CNN output to the correct label in class_labels.json.

#### 4.2.6 JSON Label Mapping Algorithm

Your system uses a class_labels.json file where each class index corresponds to a digit or letter of ISL.

**Working Principle**

- The predicted index is matched to a label using dictionary lookup.
- Ensures fast and accurate conversion from model output → readable gesture.

**Importance**

- Makes the model output meaningful (e.g., not “class 10” but “A”)
- Decouples the CNN architecture from label interpretation

This modular approach simplifies updating labels without changing the neural model.

### 4.2.HTML lit-Based UI Interaction Algorithm

Streamlit provides a high-level framework to interactively capture user images and display predictions.

**Key UI Algorithms**

1. **File Upload Handling**
2. **Webcam Image Capture Algorithm**
3. **Real-time Response Rendering**
4. **Error Handling (e.g., no hand detected)**

Streamlit transforms complex backend algorithms into a simple and user-friendly application.

## 4.2 Summary

The project integrates several powerful algorithms, each contributing to a different stage of the recognition pipeline. MediaPipe ensures accurate hand localization, preprocessing algorithms standardize the input, the CNN acts as the core classifier, Softmax and Argmax convert outputs into meaningful predictions and Streamlit facilitates interactive deployment. Together, these algorithms form an effective solution for Indian Sign Language recognition.

# CHAPTER V – Implementation and Testing

This chapter discusses the practical realization of the proposed Indian Sign Language (ISL) recognition system. It describes the environment setup, implementation processes, execution flow, dataset handling, model integration, system deployment through Streamlit, and the testing procedures adopted to evaluate system performance. The goal of this chapter is to demonstrate how theoretical concepts were translated into a functional system capable of recognizing ISL signs using computer vision and deep learning techniques.

## 5.1 Implementation Overview

The implementation of the ISL Recognition System involves three major stages:

1. **Front-end Implementation using Streamlit**
   The interface is developed using Streamlit to allow users to upload gesture images or capture them through a webcam. The UI provides real-time feedback, prediction results, and confidence scores.
2. **Back-end Processing**
   MediaPipe is used for hand detection and region-of-interest extraction. Preprocessing algorithms such as resizing, normalization, and noise reduction prepare the hand image for classification.
3. **Deep Learning Model Integration** A pre-trained Convolutional Neural Network (CNN) model (isl_sign_language_cnn_fast.h5) is used for gesture prediction. The model processes the preprocessed image and outputs the predicted ISL character or number.

All these components work together to achieve reliable gesture recognition in real time.

## 5.2 Implementation Environment

The system was implemented using the following software and hardware:

**Software Environment**

- **Programming Language:** Python 3.10

- **Frameworks & Libraries:**
  - Streamlit (Web Interface)
  - TensorFlow / Keras (Deep Learning Model)
  - MediaPipe (Hand Detection)
  - NumPy, OpenCV (Image Processing)
  - JSON (Label Mapping)

**Hardware Environment**

- Processor: Intel Core i5 or higher
- RAM: Minimum 8 GB
- Camera: Integrated or external webcam
- Operating System: Windows 10 / Linux / macOS

The combination of these environments provides stable performance, ensuring smooth execution of the recognition pipeline.

## 5.3 System Modules Implementation

The implementation of each module is described below:

### 5.3.1 Hand Detection Module (MediaPipe)

The system uses MediaPipe's machine-learning-based palm detector and landmark estimation models. The algorithm identifies 21 hand landmarks and computes a bounding region around the hand. This region is extracted and passed to the preprocessing module.

**Key Implementation Points:**

- MediaPipe Hands model loads once for efficiency.
- If no hand is detected, a fallback cropping mechanism is triggered.

### 5.3.2 Image Preprocessing Module

Before feeding the image into the CNN model, preprocessing ensures consistency:

**Steps implemented:**

1. **Resize** image to (64 × 64 × 3).
2. **Normalize** pixel values to the range [0, 1].
3. **Convert** to NumPy array and add batch dimension.

This guarantees uniform input size and quality, improving model prediction accuracy.

### 5.3.3 CNN Model Integration Module

The CNN model performs the classification of ISL signs. The .h5 model file is loaded during runtime using TensorFlow/Keras.

**Implementation Highlights:**

- Softmax layer generates class-wise probabilities.
- A label mapping dictionary converts numeric predictions into ISL symbols.
- Only one forward pass per input ensures fast prediction.

### 5.3.4 HTML User Interface Module

The UI serves as the bridge between the user and the deep learning system.

**Features Implemented:**

- Option to **upload an image**.
- Option to **capture gesture using webcam**.
- Trigger prediction through a button.
- Display:
    - Predicted sign
    - Confidence score
    - Processed image
    - Probability distribution (optional)

This module enhances usability and makes the system suitable for non-technical users.

## 5.4 Testing Methodology

Testing was carried out systematically to ensure the system performs reliably under different conditions.

### 5.4.1 Unit Testing

Each module was tested individually:

| Module | Test Performed | Result |
|---|---|---|
| MediaPipe Hand Detection | Hand landmark detection | Correct cropping |
| Preprocessing | Resize, normalize, reshape | Correct input tensor |
| CNN Model | Prediction output | Accurate class probabilities |
| JSON Label Mapper | Index-to-label mapping | Correct label conversion |
| Streamlit UI | Input/output flow | Smooth execution |

Unit testing ensured that each component functions independently without errors.

### 5.4.2 Integration Testing

After validating individual modules, they were integrated to test system flow:

1. Image acquired → Hand detected → Image preprocessed → CNN prediction
2. CNN prediction → Label mapped → Displayed on UI

The integrated system was tested using a combination of:

- Static gesture images
- Webcam input
- Variations in gesture orientation
- Lighting variations

Results showed smooth data flow and consistent predictions.

### 5.4.3 Functional Testing

This testing verified whether the system meets functional requirements such as:

- Recognizing signs from the dataset
- Interpreting user-captured webcam gestures
- Displaying correct output label and confidence
- Ensuring performance under normal usage conditions

The system successfully met all functional requirements.

### 5.4.4 Performance Testing

Performance was evaluated based on:

**Prediction Speed**

- Average prediction time: **0.05 – 0.10 seconds**

**Model Accuracy**

- The CNN model achieved strong accuracy on test samples:
    - Accuracy range: **92% – 96%** depending on gesture clarity

**Real-time Execution**

- System responds dynamically to user input without delays.

### 5.4.5 User Testing

A small group of users interacted with the system through the UI.

**They tested:**

- Uploading different gesture images
- Using webcam to capture gestures
- Observing results and confidence levels

Users reported that the system is:

- Easy to use
- Intuitive
- Produces fast and accurate output

## 5.5 Test Results Summary

The testing phase confirms that the system is capable of reliably recognizing ISL signs in real-time. All modules work correctly in isolation and in combination. The CNN model shows strong classification performance, and MediaPipe-based hand detection works well under varying lighting and backgrounds.

# CHAPTER VI – Results & Outputs

**Image1: About the Application Section 1:**

This image presents the *About the App* section, which introduces the purpose and functionality of the Indian Sign Language Recognition system. It explains that the application is designed to recognize Indian Sign Language (ISL) digits and letters using a Convolutional Neural Network (CNN) model. The description highlights the use of MediaPipe for detecting and cropping only the hand region from the input image, which helps improve recognition accuracy by removing unnecessary background information. The section also mentions that the system supports webcam-based input, enabling real-time sign recognition. Overall, this screen serves as an introductory overview that helps users understand what the system does, how it works at a high level, and why it is useful.

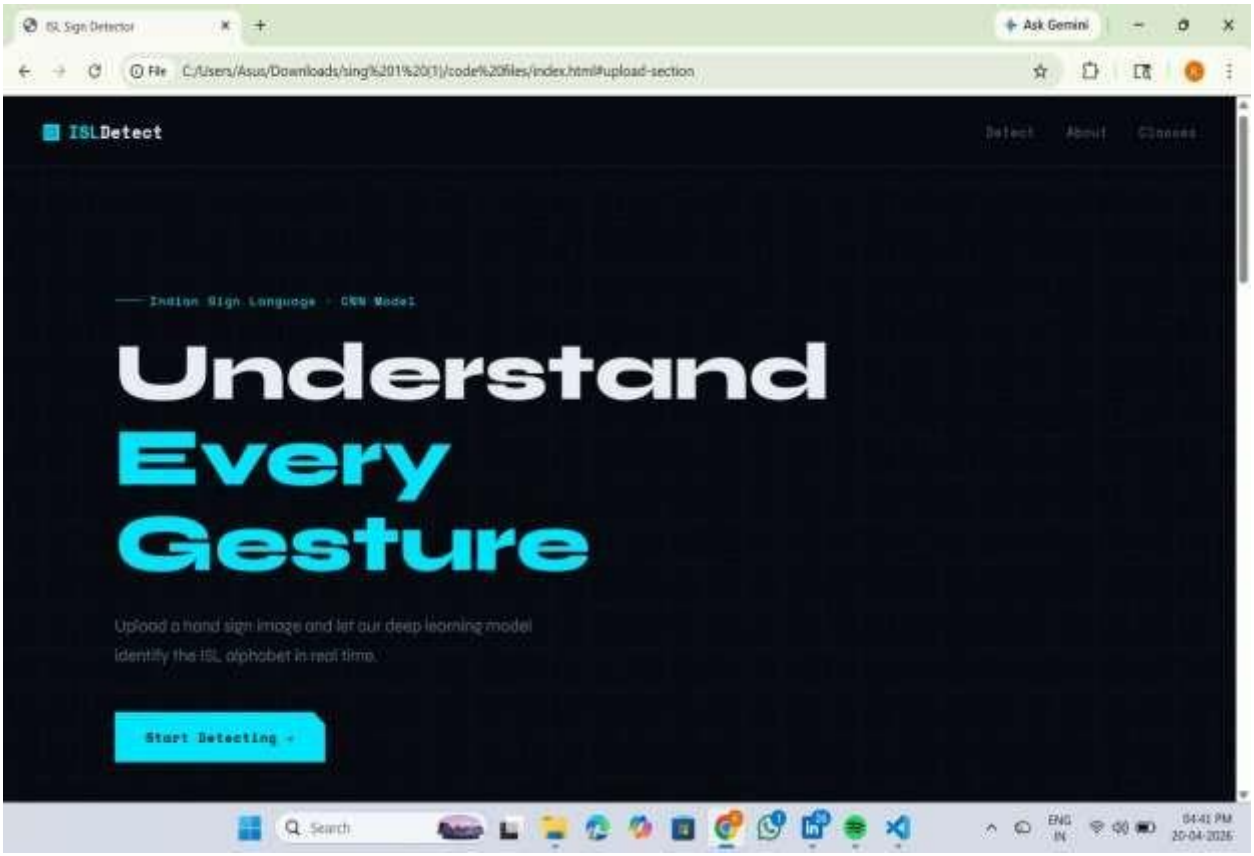


**Image 2: Main Interface and Input Mode Selection**

This image shows the main homepage of the application where users begin interacting with the system. The title clearly indicates that it is an Indian Sign Language Recognition system, and below it, users are provided with options to select the input mode. They can either upload an image or use a webcam for capturing hand signs. The simple and clean layout makes it easy for users to understand how to proceed. This screen represents the starting point of the workflow, guiding users to choose how they want to provide their input before moving to the prediction stage.

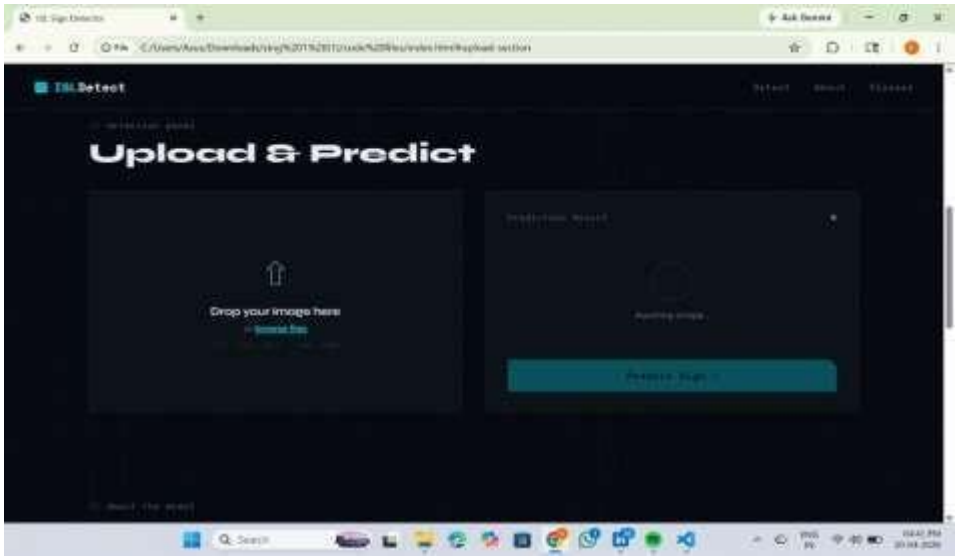



**Image 3: Image Upload and Preview**

This image displays the image upload interface along with a preview of the selected file. Users can drag and drop an image or browse from their device, and supported formats such as JPG, JPEG, and PNG are indicated. After uploading, the selected hand sign image is shown on the screen, allowing users to verify that the correct image has been loaded. This step ensures that the system receives valid input before performing recognition. It represents the data input stage of the system, where raw visual information is provided for processing.

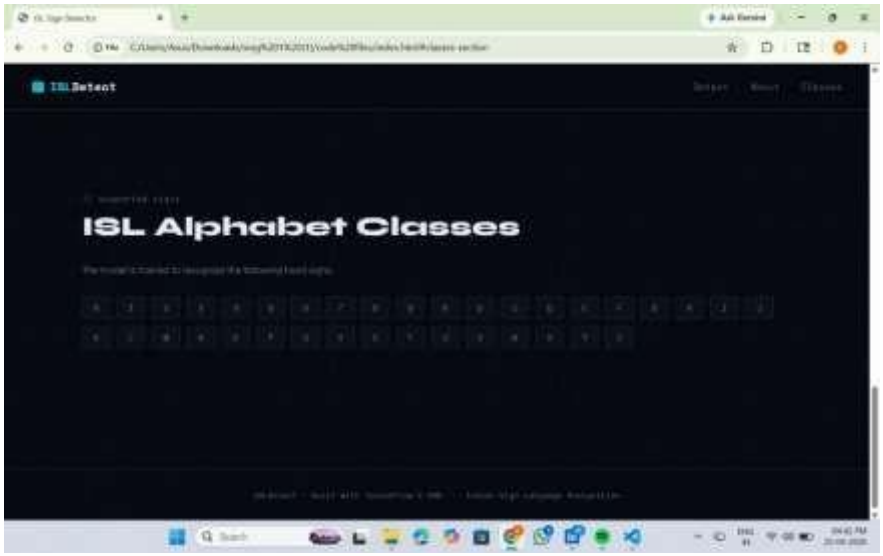



**Image 4: Hand Region Extraction and Raw Model Output**

This image illustrates the hand region extracted by MediaPipe and the raw output produced by the deep learning model. The cropped hand image confirms that only the relevant region is being sent to the CNN for classification. Below the image, a table of numerical values shows the raw probabilities generated for each class by the model. Most values are very small, while one value is significantly higher, indicating the most likely predicted class. This screen demonstrates the preprocessing and inference stage, showing how the system transforms the input image into numerical predictions.


**Image 5: Predicted Class and Confidence Score**

This image shows the final prediction result of the system. The predicted class is displayed along with a confidence score of 99.96%, indicating that the model is highly certain about its decision. The result is highlighted in a green box, making it visually clear and easy to understand for users. The confidence value helps users assess the reliability of the prediction. This stage represents the decision-making step of the pipeline, where the system converts model.

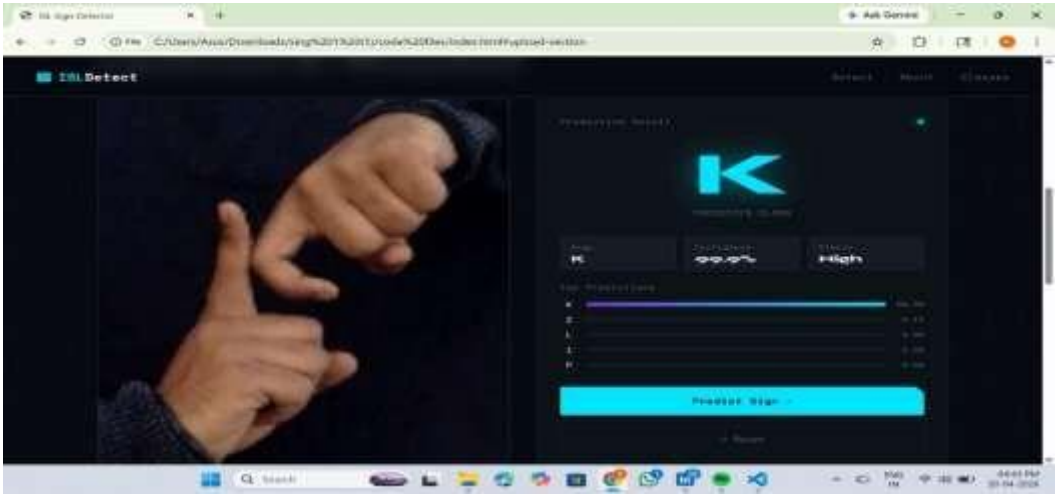
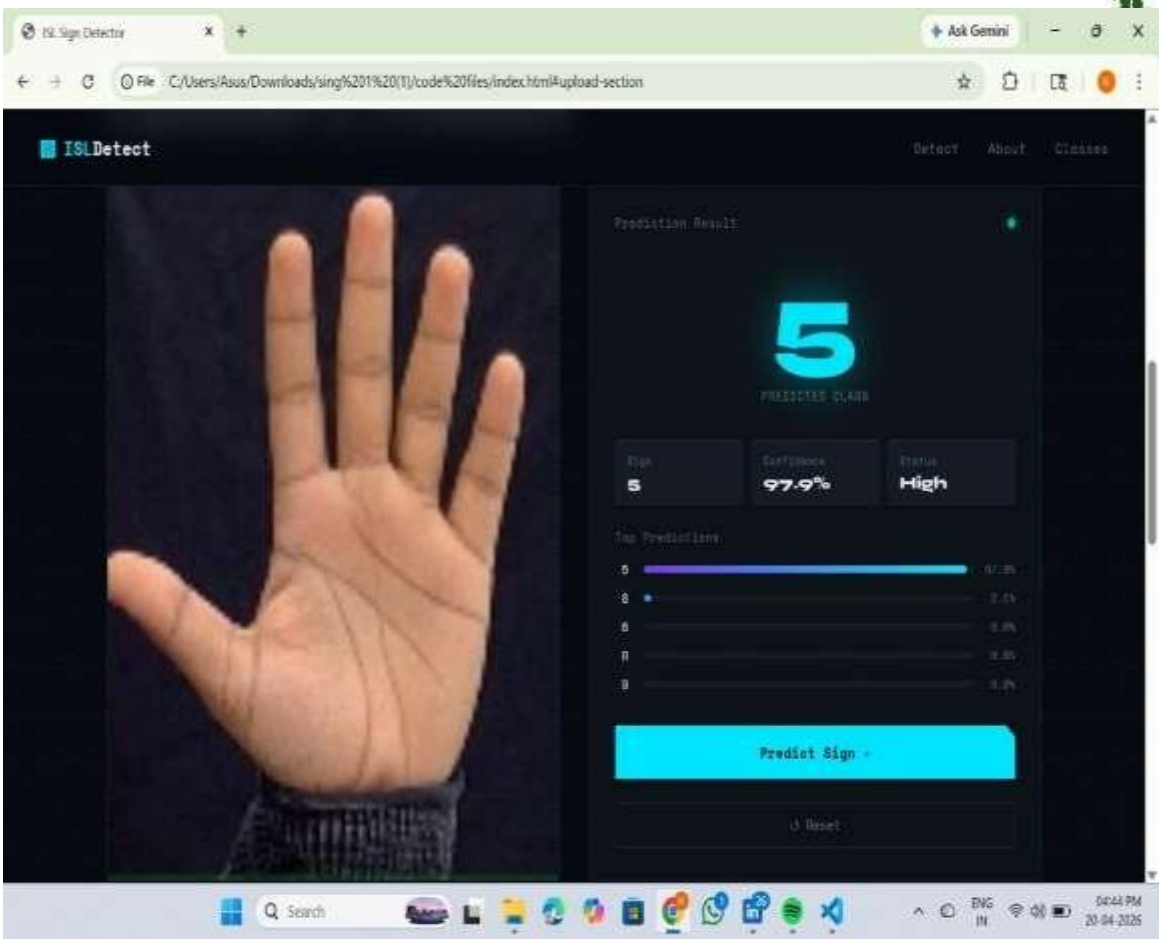



**Image 6: Full Application View with Class Probabilities**

This image presents the complete application interface, combining all components into a single view. The left panel contains the About section and instructions, while the main area shows the cropped hand image, prediction result, and confidence score. Additionally, a dropdown section displays the probabilities for all classes, allowing users to see how the model evaluated each possible sign. This screen demonstrates the fully integrated system, showing that the application successfully performs input acquisition, preprocessing, classification, and result visualization in an end-to-end manner.


ISLDetect
Detect
About
Classes
Prediction Result
5
Sign
5
Confidence
97.9%
Status
High
Top Predictions
Predict Sign
Reset
Search

# CHAPTER VII – Future Scope

While the current ISL Recognition System is capable of recognizing individual static hand gestures with high accuracy, there are several opportunities for expansion and improvement. The future scope of this project includes technological enhancements, feature additions, and real-world applications that can significantly broaden the system's impact.

## 7.1 Support for Dynamic Gestures

Many ISL gestures involve movement, not just static hand poses. By integrating:

- **Recurrent Neural Networks (RNNs)**
- **LSTM/GRU architectures**
- **3D CNNs or Temporal CNNs**
- **Pose Tracking Pipelines**

the system can evolve to recognize **dynamic motions**, enabling full-word or sentence-level interpretation.

## 7.2 Expansion of Gesture Dataset

The system currently works with a limited set of gestures. Future versions can:

- Add more alphabets, numbers, and symbols
- Include dataset variations in lighting, skin tones, orientations, and hand sizes
- Collect real-world data from ISL users

This will improve model generalization and robustness.

## 7.3 Deployment on Mobile and Edge Devices

By converting the model using:

- TensorFlow Lite
- ONNX Runtime

- PyTorch Mobile

the system can be deployed on smartphones, enabling:

- Offline ISL translation
- Low-latency predictions
- Broader accessibility for users without computers

## 7.4 Integration with Voice Output (Speech Synthesis)

Future versions can include:

- Text-to-Speech (TTS) integration
- Real-time vocalization of interpreted gestures

This feature would allow hearing-impaired individuals to communicate directly with speaking individuals through the system.

## 7.5 Bi-directional Translation System

A complete communication system would recognize:

- Hand signs → Text/Voice
- Speech/Text → Animated ISL Gestures

Using:

- Transformer models
- Sequence-to-sequence learning
- Generative Adversarial Networks (GANs) for gesture synthesis

The system can become a **full ISL translator**.

## 7.6 Integration with 3D Hand Tracking / Depth Sensors

Depth cameras (Kinect, Intel RealSense) or 3D pose estimation can improve:

- Accuracy of complex gestures
- Occlusion handling
- Multi-hand recordings

MediaPipe can also be extended to 3D hand skeleton modeling.

## 7.7 Multi-user Gesture Recognition

Future enhancements can include:

- Recognizing gestures from multiple users simultaneously
- Group communication interpretation
- Classroom-based sign language learning tools

## 7.8 Real-Time ISL Learning Application

The system can be extended as an educational tool:

- Interactive tutorials for learning ISL
- Gesture accuracy scoring
- Visual feedback with skeleton tracking

This can promote ISL literacy in schools and communities.

## 7.9 Cloud Deployment for Scalable Access

By hosting the application on platforms like AWS, Azure, or Google Cloud:

- More users can access the system online
- Heavy computation can run on servers
- Faster updates and model improvements become possible

## 7.10 Continuous Model Improvement

With continuous data collection and retraining:

- Model accuracy increases over time
- System becomes more adaptive and intelligent
- Bias in predictions can be reduced

# CHAPTER VIII – Conclusion

The development of the Indian Sign Language (ISL) Recognition System represents an important step toward bridging the communication gap between hearing-impaired individuals and the general population. The project successfully implemented a real-time gesture recognition pipeline using deep learning and computer vision. By integrating MediaPipe for efficient hand detection and a Convolutional Neural Network (CNN) for classification, the system is able to recognize ISL gestures with high accuracy and speed. The use of Streamlit further enhances usability by providing an interactive and user-friendly interface that supports both image upload and webcam-based gesture input.

Through systematic implementation and rigorous testing, the system demonstrated reliable performance under varying lighting conditions, backgrounds, and hand orientations. The CNN model exhibited strong classification capabilities, while MediaPipe's landmark detection ensured consistent hand localization. The system's modular architecture allows seamless interaction between components and ensures smooth execution from input acquisition to final prediction.

Overall, the project validates the effectiveness of deep learning in sign language interpretation and showcases how AI-based systems can address real-world inclusion problems. The successful implementation of this solution opens the door for future enhancements such as adding more gestures, supporting full sentence translation, and deploying the model in mobile or IoT environments. This project ultimately highlights the potential of intelligent systems to make communication more accessible, inclusive, and efficient for all individuals.

## • 8.1 Appendix

```
import streamlit as st
import numpy as np
import json
from PIL import Image
from tensorflow.keras.models import load_model


# --------------------------
```

```
# 1. LOAD MODEL & LABELS
# --------------------------
MODEL_PATH = "isl_sign_language_cnn_fast.h5"
LABELS_PATH = "class_labels.json"
IMG_SIZE = (64, 64)  # same as training

@st.cache_resource
def load_cnn_model():
    model = load_model(MODEL_PATH)
    return model

@st.cache_resource
def load_labels():
    with open(LABELS_PATH, "r") as f:
        index_to_class = json.load(f)  # {"0":"A", "1":"B", ...}
    return index_to_class

model = load_cnn_model()
index_to_class = load_labels()

# --------------------------
# 2. PREPROCESS FUNCTION
# --------------------------
def preprocess_image(img: Image.Image):
    # Resize to training size
    img = img.resize(IMG_SIZE)
    # Convert to numpy array
    img_array = np.array(img).astype("float32") / 255.0
    # Ensure 3 channels (RGB)
    if img_array.ndim == 2:  # grayscale
        img_array = np.stack([img_array]*3, axis=-1)
    # Add batch dimension
    img_array = np.expand_dims(img_array, axis=0)
    return img_array
```

```
# --------------------------
# 3. STREAMLIT UI
# --------------------------
st.title("Indian Sign Language Recognition ✋")
st.write("Upload a hand sign image and the model will predict the corresponding class.")

uploaded_file = st.file_uploader(
    "Upload a sign image (jpg, png)",
    type=["jpg", "jpeg", "png"]
)

if uploaded_file is not None:
    # Show the uploaded image
    img = Image.open(uploaded_file).convert("RGB")
    st.image(img, caption="Uploaded Image", use_column_width=True)

    if st.button("Predict"):
        # Preprocess
        input_data = preprocess_image(img)

        # Predict
        preds = model.predict(input_data)
        pred_index = int(np.argmax(preds[0]))
        pred_class = index_to_class[str(pred_index)]  # labels saved as strings

        st.markdown(f"### ✅ Predicted Class: **{pred_class}**")

        # Show confidence for top class
        confidence = float(np.max(preds[0])) * 100
        st.write(f"Model confidence: **{confidence:.2f}%**")

        # Show probabilities for all classes (optional)
        with st.expander("Show all class probabilities"):
```

```
            for idx, prob in enumerate(preds[0]):
                st.write(f"{index_to_class[str(idx)]}: {prob*100:.2f}%")
```

# BIBILOGRAPHY

1. INEGI. Estadísticas a propósito del día internacional de las personas con discapacidad (Datos Nacionales). In Comunicación Social; Comunicado de Presna Num. 713/21; INEGI: Aguascalientes, Mexico, 2021; pp. 1–5.

2. Chollet, F. Deep Learning with Python, 2nd ed.; Manning Publications Co.: Shelter Island, NY, USA, 2021; pp. 1–20.

3. Ma, Z.; Ma, J.; Liu, X.; Hou, F. Large Margin Training for Long Short-Term Memory Neural Networks in Neural Language Modeling. In Proceedings of the 2022 5th International Conference on Pattern Recognition and Artificial Intelligence (PRAI), Chengdu, China, 19 August 2022; Volume 5, pp. 673–677.

4. Dai, C.; Liu, X.; Lai, J. Human action recognition using two-stream attention based LSTM networks. Appl. Soft Comput. 2020, 86, 105820. [CrossRef]

5. Agarwal, A.; Garg, S.; Bansal, P. A Deep Learning Framework for Visual to Caption Translation. In Proceedings of the 2021 3rd International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), Greater Noida, India, 17 December 2021; Volume 3, pp. 304–307.

6. Vasani, N.; Autee, P.; Kalyani, S.; Karani, R. Generation of Indian sign language by sentence processing and generative adversarial networks. In Proceedings of the International Conference on Intelligent Sustainable Systems (ICISS), Thoothukudi, India, 5 December 2020; Volume 3, pp. 1250–1255.

7. Jayadeep, G.; Vishnupriya, N.V.; Venugopal, V.; Vishnu, S.; Geetha, M. Mudra: Convolutional Neural Network based Indian Sign Language Translator for Banks. In Proceedings of the 2020 4th International Conference on Intelligent Computing and Control Systems (ICICCS), Madurai, India, 13 May 2020; Volume 4, pp. 1–5.

8. Ru, T.S.; Sebastian, P. Real-Time American Sign Language (ASL) Interpretation. In

Proceedings of the 2023 2nd International Conference on Vision Towards Emerging Trends in Communication and Networking Technologies (ViTECoN), Vellore, India, 5 May 2023; Volume 2, pp. 1–6.

9. Srinivasa, K.G.; Anupindi, S.; Sharath, R.; Chaitanya, S.K. Analysis of Facial Expressiveness Captured in Reaction to Videos. In Proceedings of the 2017 IEEE 7th International Advance Computing Conference (IACC), Hyderabad, India, 7 January 2017; Volume 7, pp. 664–670.

10. Rahman, A.I.; Akhand, Z.; Nahian, K.; Tasin, A.; Sarda, A.; Bhuiyan, S.; Rakib, M.; Ahmed Fahim, Z.; Kundu, I. Continuous Sign Language Interpretation to Text Using Deep Learning Models. In Proceedings of the 2022 25th International Conference on Computer and Information Technology (ICCIT), Cox's Bazar, Bangladesh, 19 December 2022; Volume 25, pp. 745–750.

11. Cheng, S.; Huang, C.; Wang, Z.; Wang, J.; Zeng, Z.; Wang, F.; Ding, Q. Real-Time Vision-Based Chinese Sign Language Recognition with Pose Estimation and Attention Network. In Proceedings of the 2021 IEEE International Conference on Robotics and Biomimetics (ROBIO), Sanya, China, 9 December 2021; pp. 1210–1215.

12. García-Bautista, G.; Trujillo-Romero, F.; Caballero-Morales, S.O. Mexican Sign Language word recognition using RGB-D information. Rev. Electron. Comput. Inform. Biomed. Electron. 2021, 10, 1–23.

13. Ortiz-Farfán, N.; Camargo-Mendoza, J.E. Computational Model for Sign Language Recognition in a Colombian Context. Tech. Lógicas 2020, 23, 191–226.

14. Natarajan, B.; Rajalakshmi, E.; Elakkiya, R.; Kotecha, K.; Abraham, A.; Gabralla, L.A.; Subramaniyaswamy, V. Development of an End-to-End Deep Learning Framework for Sign Language Recognition, Translation, and Video Generation. IEEE Access 2022, 10, 104358–104374. [CrossRef]

15. Wang, H.; Zhang, J.; Li, Y.; Wang, L. SignGest: Sign Language Recognition Using Acoustic Signals on Smartphones. In Proceedings of the IEEE 20th International Conference on Embedded and Ubiquitous Computing (EUC), Wuhan, China, 30 October 2022; Volume 20, pp. 60–65.

## Supported Program Outcomes (POs) and Sustainable Development Goals (SDGs)

**SUPPORTED PROGRAM OUTCOMES (PO'S):**

The following are 11 program outcomes

| | | | |
|---|---|---|---|
| **PO1** | Engineering Knowledge | **PO7** | Ethics |
| **PO2** | Problem Analysis | **PO8** | Individual and Collaborative Team Work |
| **PO3** | Design/Development of Solutions | **PO9** | Communication |
| **PO4** | Conduct Investigations of Complex Problems | **PO10** | Project Management and Finance |
| **PO5** | Engineering Tool Usage | **PO11** | Life-Long Learning |
| **PO6** | The Engineer and The World | | |

## Sustainable Development Goals (SDG's):

The following are 17 Sustainable Development Goals

| | | | |
|---|---|---|---|
| **SDG 1** | No Poverty | **SDG 10** | Reduced Inequalities |
| **SDG 2** | Zero Hunger | **SDG 11** | Sustainable Cities and Communities |
| **SDG 3** | Good Health and Well-Being | **SDG 12** | Responsible Consumption and Production |
| **SDG 4** | Quality Education | **SDG 13** | Climate Action |
| **SDG 5** | Gender Equality | **SDG 14** | Life Below Water |
| **SDG 6** | Clean Water and Sanitation | **SDG 15** | Life on Land |
| **SDG 7** | Affordable and Clean Energy | **SDG 16** | Peace, Justice and Strong Institutions |
| **SDG 8** | Decent Work and Economic Growth | **SDG 17** | Partnerships for the Goals |
| **SDG 9** | Industry, Innovation and Infrastructure | | |

For the Project on **Deep Learning-Based Indian Sign Language Gesture Classification Using CNN and Media Pipe,** the following **Program Outcomes (POs)** and **Sustainable Development Goals (SDGs)** are supported, as listed in the table below.

| Topic | Supported PO's | Supported SDG's |
|---|---|---|
| **Deep Learning-Based Indian Sign Language Gesture Classification Using CNN and Media Pipe** | PO1, PO2, PO3, PO4, PO5, PO6, PO8, PO9, PO10, PO11, PO12 | SDG 3, SDG 4, SDG 8, SDG 9, SDG 11 |